PROJECT REPORT

Submitted by- SARIF AHMAD

a) Design & Verification of 8-bit Arithmetic Logic Unit (ALU) by using the System Verilog.

b) Tools Used: Synopsys VCS and Verdi, Design Compiler.

c) Verify it through the System Verilog testbenches.

d) Synthesized the design by Synopsys Design compiler tool (SDC) at 32nm technology node and generated the Mapping netlist file.

1.0 Abstract

This project presents the design and functional verification of an **8-bit** Arithmetic Logic Unit (ALU) with flexible, parameterized specifications for `DATA_WIDTH` using System Verilog. The ALU performs a variety of arithmetic logical, and shifting operations including addition, subtraction, AND, OR, XOR, and comparison. It also includes overflow detection and is verified using a clock-driven testbench that tests multiple operation modes. The ALU design circuit that takes two operands and operation code as input. The design is written in synthesizable System Verilog suitable for RTL synthesis using Synopsys Design Compiler. The complete design has been simulated and verified using a System Verilog testbench in Synopsys VCS and debugged using the Verdi tool. Furthermore, the design was fully synthesized using the Synopsys Design Compiler (DC) at the **32 nm** technology node, and a mapped netlist was successfully generated and also analysed the quality of report, timing, area, and power report. The 8-bit ALU demonstrates how digital systems process and manipulate data, making, it a cornerstone in the design of CPUs, microcontrollers, and other computational hardware.

2.0 Introduction

An **ALU**, or **Arithmetic Logic Unit**, is a most important fundamental digital circuit within a computer's Central Processing Unit (CPU) that performs arithmetic and logical operations on binary numbers. It acts as the "calculator" of the CPU, handling tasks like addition, subtraction, AND, OR, and comparisons. The ALU takes in input data (operands) and an instruction code, performs the requested operation, and sends the result to be stored in memory or a register. An ALU is the **heart of any processor** or digital system. It performs

arithmetic and logic operations on input operands. In this design, I implemented an 8-bit ALU in system Verilog., capable of executing the following operations based on 4-bit opcode.

Core functions:

- **Arithmetic operations:**

  The ALU performs mathematical calculations, such as addition, subtraction, multiplication, and division.

- **Logical operations:**

  It handles logical and bitwise operations, including comparisons (e.g., greater than, less than, equal to), AND, OR, and NOT.

- **Bitwise Operations:** Manipulates binary numbers at the bit level (shifting and rotating bits).
- **Comparison**: Equality, greater than, less than.

- **Data manipulation:** The ALU processes binary data to execute instructions and perform decision-making operations within the CPU.
- **Executes instructions**: Works with the Control Unit to execute instructions by processing the data that the Control Unit directs it to.
- **Outputs result**: Sends the result of its operations to the computer's memory or registers

How it works:

- **Inputs:**

  The ALU receives data, known as operands, from registers (temporary storage locations).

- **Control Signals:**

  The CPU's control unit sends instructions to the ALU to tell it which operation to perform on the operands.

- **Processing:**

  The ALU executes the operation, and the result is stored in an output register.

- **Flags:**

  The ALU also generates "status flags" that indicate information about the result, such as whether it was zero, a carry occurred, or if it was negative. These flags are stored in a status register.

- **Functionality**:

  The ALU can perform a variety of operations, including:

- **Arithmetic**: Addition, subtraction, increment, decrement.
- **Logical**: AND, OR, XOR, XNOR.
- **Comparison**: Equality, greater than, less than.
- **Bit-wise**: Shifting and rotating bits.

- **Implementation**:

  This project can be implemented using hardware description languages like VHDL or Verilog, which allow for the design of complex digital circuits that can then be synthesized onto hardware like FPGAs.

- **Testing**: The design must be tested to verify that it performs all the specified operations correctly. This often involves creating test benches to simulate different inputs and operations.

3.0 Project objectives

- ✓ To design a parameterized 8-bit ALU capable of handling basic arithmetic and logic operations.
- ✓ Explore how different control signals (opcode) influence the results.
- ✓ To verify its functionality through a System Verilog testbench using both directed and random test vectors.
- ✓ To implement overflow detection logic for arithmetic operations.
- ✓ To ensure synthesizability and compatibility with standard ASIC design tools.
- ✓ Validate the design using Synopsys VCS and Verdi tools.
- ✓ Synthesize the RTL design using Synopsys Design compiler (DC) at 32nm.

Design Description and functional Features: -

Inputs: - `A`, `B` → 8-bit operands

Opcode: - `OP` → 3-bit operation code

Outputs: -

- `RESULT` → 8-bit result of the selected operation
- `overflow` → Overflow flag (for addition/subtraction)
- Supported Operations: -

Operation code	Function	Description
0000	Addition	A+B
0001	Subtraction	A-B
0010	Bitwise AND	A & B
0011	Bitwise OR	A | B
0100	Bitwise XOR	A ^ B
0101	Bitwise NOT	~ A

0110	Bitwise NAND	~ (A & B)
0111	Bitwise NOR	~ (A | B)
1000	LEFT SHIFT	<<< 1
1001	RIGHT SHIFT	>>> 1
1010	INCREMENT	A +1
1011	DECREMENT	A -1
1100	Greater COMPARISION	A > B
1101	EQUALITY comp	A == B
1110	LEFT SHIFT	A << 2
1111	BUFFER	B

- **Overflow Detection Logic:** Overflow occurs when the result of addition/subtraction exceeds the representable range of 8 bits.

Parameterized design for flexibility: The module supports customizable data width to suit various application requirements.

Synchronous operation: Input and output operations are synchronized with the clock signal, ensuring reliable data handling.

Parameters: - DATA_WIDTH are defined to make the module reusable for different ALU operation modes.

Operation: -The first always @ (posedge clk) block handles all operations of ALU. When en is high. The second always @ (posedge clk) block provides the output at output terminal.

Synchronous Operation: -

All operations are triggered on the positive edge of the clock. clk=100 MHz used, ensuring reliable operation modes.

Here, **8-bit ALU means** DATA_WIDTH is **8-bits** means each data is of 8-bits**.**

clk: Positive edge triggering for synchronizing the operations.

rst: Active high reset signal to initialize the output.

A, B: input data of 8-bit.

en: enable signal. (en=1) means data is ready for operation.

Output signal: output result (8-bits), overflow.

Opcode: use for the selection for the operation (4-bits)

Design Specifications: -

Here, we designed a synthesizable **8-bit ALU**. The design includes **input pins** like clk, rst, en, and an **8-bit** of **data input A & B,** along with **output pins** like the **8-bit data output result**, **overflow (1-bit) and** followed by the corresponding opcode signals of 4-bits. Finally, the design has been **verified through testbenches** that cover at all possible scenarios.

RTL Design Code (Synthesizable): -

File name: alu_8_bit.sv

```
// Design the System Verilog code of ALU-8-bit
module ALU_8_bit #(parameter DATA_WIDTH = 8)(
        input logic clk,
        input logic rst, en,
        input logic signed [DATA_WIDTH-1:0] A,
      input logic signed [DATA_WIDTH-1:0] B,
      input logic [3:0] opcode,
      output logic signed [DATA_WIDTH-1:0] result,
        output logic overflow);

logic signed [DATA_WIDTH-1:0] opA_reg, opB_reg, res_reg;
logic [3:0] opcode_reg;
logic ov_reg, overtemp;


always_ff @(posedge clk or posedge rst)begin
        if (rst) begin
                opA_reg <= '0;
                opB_reg <= '0;
                opcode_reg <= '0;
        end
```

```
        else if (en) begin
                opA_reg <= A;
                opB_reg <= B;
                opcode_reg <= opcode;
        end
end

always_comb begin
        ov_reg  = 0;
        res_reg = '0;
        overtemp=0;

        unique case (opcode_reg)
        4'd0: begin
                res_reg = opA_reg + opB_reg;
                overtemp = (~opA_reg[DATA_WIDTH-1] & ~opB_reg[DATA_WIDTH-1]
& res_reg[DATA_WIDTH-1]) |
                ( opA_reg[DATA_WIDTH-1] &  opB_reg[DATA_WIDTH-1] &
~res_reg[DATA_WIDTH-1]);
          if (overtemp) ov_reg = 1;
   end

4'd1: begin
        res_reg = opA_reg - opB_reg;
        overtemp = (~opA_reg[DATA_WIDTH-1] & opB_reg[DATA_WIDTH-1] &
res_reg[DATA_WIDTH-1]) |
                ( opA_reg[DATA_WIDTH-1] & ~opB_reg[DATA_WIDTH-1] &
~res_reg[DATA_WIDTH-1]);
```

```
            if (overtemp) ov_reg = 1;
   end
4'd2: res_reg = opA_reg & opB_reg;
4'd3: res_reg = opA_reg | opB_reg;
4'd4: res_reg = opA_reg ^ opB_reg;
4'd5: res_reg = ~opA_reg;
4'd6: res_reg = ~(opA_reg & opB_reg);
4'd7: res_reg = ~(opA_reg | opB_reg);
4'd8: res_reg = opA_reg <<< 1;
4'd9: res_reg = opA_reg >>> 1;
4'd10:begin
          res_reg = opA_reg +1;
          if (opA_reg == {1'b0, {(DATA_WIDTH-1){1'b1}}}) ov_reg=1;
end
4'd11:begin
          res_reg = opA_reg -1;
        if (opA_reg == {1'b1, {(DATA_WIDTH-1){1'b0}}}) ov_reg=1;
end
4'd12:res_reg = (opA_reg > opB_reg)? {{(DATA_WIDTH-1){1'b0}},1'b1}:'0;
4'd13:res_reg = (opA_reg == opB_reg)? {{(DATA_WIDTH-1){1'b0}},1'b1}:'0;
4'd14:res_reg = opA_reg <<2;
4'd15:res_reg = opB_reg;
default:res_reg = {DATA_WIDTH{1'bx}};
endcase
end

always_ff @(posedge clk or posedge rst)begin
```

```
        if (rst) begin
                result <= '0;
                overflow <= 1'b0;
        end
        else if (en) begin
                result <= res_reg;
                overflow <= ov_reg;
        end
end
endmodule
```

Testbench

File name: alu_8_bit_tb.sv

```
// Testbench of parameterized 8-bit ALU
module tb_alu_8_bit ();
parameter DATA_WIDTH=8;
logic clk, rst,en;
logic signed [DATA_WIDTH-1:0] A, B;
logic [3:0] opcode;
logic signed [DATA_WIDTH-1:0] result;
logic overflow;

ALU_8_bit #(.DATA_WIDTH(DATA_WIDTH))
dut(.clk(clk),.rst(rst),.en(en),.A(A),.B(B),.opcode(opcode),.result(result),.overflow(overflow));

integer i;
initial clk=0;
always #5 clk = ~clk;
```

```
initial begin
        $display ("Starting ALU test...");
        rst = 1; en=0; A = 0; B = 0; opcode = 0;
        @ (posedge clk);
//      @ (posedge clk);
        rst=0; en=1;
        $display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);


        A=8'd15; B=8'd3;
        for (i=0; i<16; i=i+1) begin
                @ (posedge clk);
                opcode=i[3:0];
                @ (posedge clk);
                #1;
                $display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);
        end


@ (posedge clk);
A = 127; B = 1; opcode = 0;
@ (posedge clk);
#1;
$display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);
//#1;
```

@ (posedge clk);

A = -128; B = 1; opcode = 1;

@ (posedge clk);

#1;

$display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);

@ (posedge clk);

A = 0; B = 0; opcode = 0;

@ (posedge clk);

#1;

$display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);

@ (posedge clk);

A = 255; B = 1; opcode = 0;

@ (posedge clk);

#1;

$display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);

@ (posedge clk);

A = 255; B = 255; opcode = 0;

@ (posedge clk);

#1;

$display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);

@ (posedge clk);

```
A = 127; opcode = 10;
@ (posedge clk);
#1;
$display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);

@ (posedge clk);
A = -128; opcode = 11;
@ (posedge clk);
#1;
$display ("Time=%0t OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);

repeat (16) begin
        @ (posedge clk);
        A = $urandom;
        B = $urandom;
        opcode = $urandom % 10;
        @ (posedge clk);
        #1;
        $display ("Time=%0t, OP: %0d | A: %0h | B: %0h | RESULT: %0h |overflow: %0b", $time, opcode, A, B, result, overflow);
end
$display ("All test cases completed successfully ");
#100 $finish ();
end

initial begin
```

```
        $fsdbDumpfile("myfile.fsdb");
        $fsdbDumpvars(0,tb_alu_8_bit);
end
endmodule
```

Run command

vcs -sverilog alu_8_bit.sv alu_8_bit_tb.sv -kdb -lca -debug_access+all -full64 && ./simv

Simulation Result

Output

At the first positive clock edge, apply inputs (happens after posedge) & next posedge DUT samples inputs into op_reg and #1 small delay so outputs settle.

Time=5 OP: 0 | A: 0 | B: 0 | RESULT: 0 |overflow: 0

Time=26 OP: 0 | A: f | B: 3 | RESULT: 12 |overflow: 0

Time=46 OP: 1 | A: f | B: 3 | RESULT: c |overflow: 0

Time=66 OP: 2 | A: f | B: 3 | RESULT: 3 |overflow: 0

Time=86 OP: 3 | A: f | B: 3 | RESULT: f |overflow: 0

Time=106 OP: 4 | A: f | B: 3 | RESULT: c |overflow: 0

Time=126 OP: 5 | A: f | B: 3 | RESULT: f0 |overflow: 0

Time=146 OP: 6 | A: f | B: 3 | RESULT: fc |overflow: 0

Time=166 OP: 7 | A: f | B: 3 | RESULT: f0 |overflow: 0

Time=186 OP: 8 | A: f | B: 3 | RESULT: 1e |overflow: 0

Time=206 OP: 9 | A: f | B: 3 | RESULT: 7 |overflow: 0

Time=226 OP: 10 | A: f | B: 3 | RESULT: 10 |overflow: 0

Time=246 OP: 11 | A: f | B: 3 | RESULT: e |overflow: 0

Time=266 OP: 12 | A: f | B: 3 | RESULT: 1 |overflow: 0

Time=286 OP: 13 | A: f | B: 3 | RESULT: 0 |overflow: 0

Time=306 OP: 14 | A: f | B: 3 | RESULT: 3c |overflow: 0

Time=326 OP: 15 | A: f | B: 3 | RESULT: 3 |overflow: 0

Time=346 OP: 0 | A: 7f | B: 1 | RESULT: 80 |overflow: 1

Time=366 OP: 1 | A: 80 | B: 1 | RESULT: 7f |overflow: 1

Time=386 OP: 0 | A: 0 | B: 0 | RESULT: 0 |overflow: 0

Time=406 OP: 0 | A: ff | B: 1 | RESULT: 0 |overflow: 0

Time=426 OP: 0 | A: ff | B: ff | RESULT: fe |overflow: 0

Time=446 OP: 10 | A: 7f | B: ff | RESULT: 80 |overflow: 1

Time=466 OP: 11 | A: 80 | B: ff | RESULT: 7f |overflow: 1

Time=486, OP: 0 | A: 51 | B: cd | RESULT: 1e |overflow: 0

Time=506, OP: 5 | A: db | B: 71 | RESULT: 24 |overflow: 0

Time=526, OP: 5 | A: e9 | B: 98 | RESULT: 16 |overflow: 0

Time=546, OP: 1 | A: a4 | B: a7 | RESULT: fd |overflow: 0

Time=566, OP: 8 | A: 0 | B: 4f | RESULT: 0 |overflow: 0

Time=586, OP: 1 | A: e7 | B: d8 | RESULT: f |overflow: 0

Time=606, OP: 5 | A: 8b | B: 7 | RESULT: 74 |overflow: 0

Time=626, OP: 0 | A: 15 | B: e6 | RESULT: fb |overflow: 0

Time=646, OP: 6 | A: 1 | B: 72 | RESULT: ff |overflow: 0

Time=666, OP: 9 | A: dc | B: d7 | RESULT: ee |overflow: 0

Time=686, OP: 0 | A: cc | B: 7c | RESULT: 48 |overflow: 0

Time=706, OP: 7 | A: 67 | B: 87 | RESULT: 18 |overflow: 0

Time=726, OP: 5 | A: 63 | B: d2 | RESULT: 9c |overflow: 0

Time=746, OP: 4 | A: 41 | B: 99 | RESULT: d8 |overflow: 0

Time=766, OP: 6 | A: 85 | B: 21 | RESULT: fe |overflow: 0

Time=786, OP: 7 | A: 67 | B: a | RESULT: 90 |overflow: 0

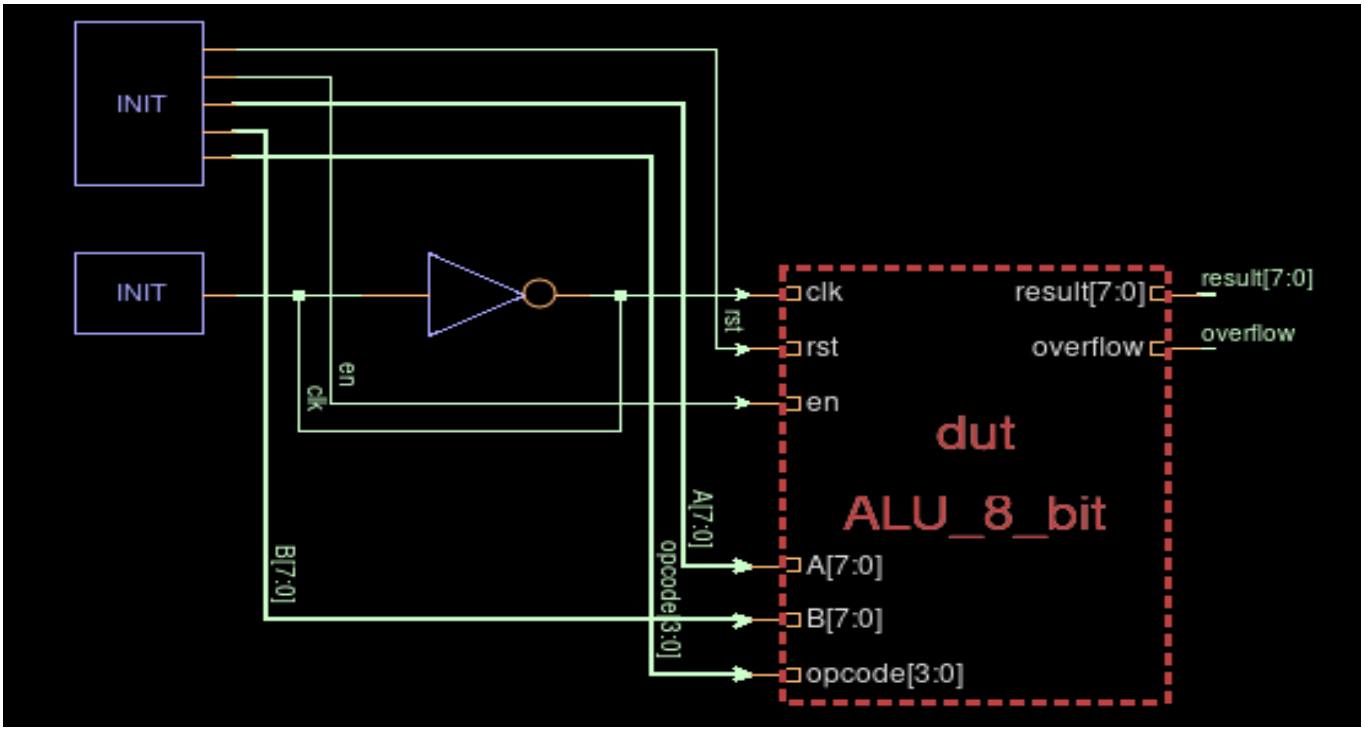
All test cases completed succesfully

$finish called from file "alu_8_bit_tb.sv", line 88.

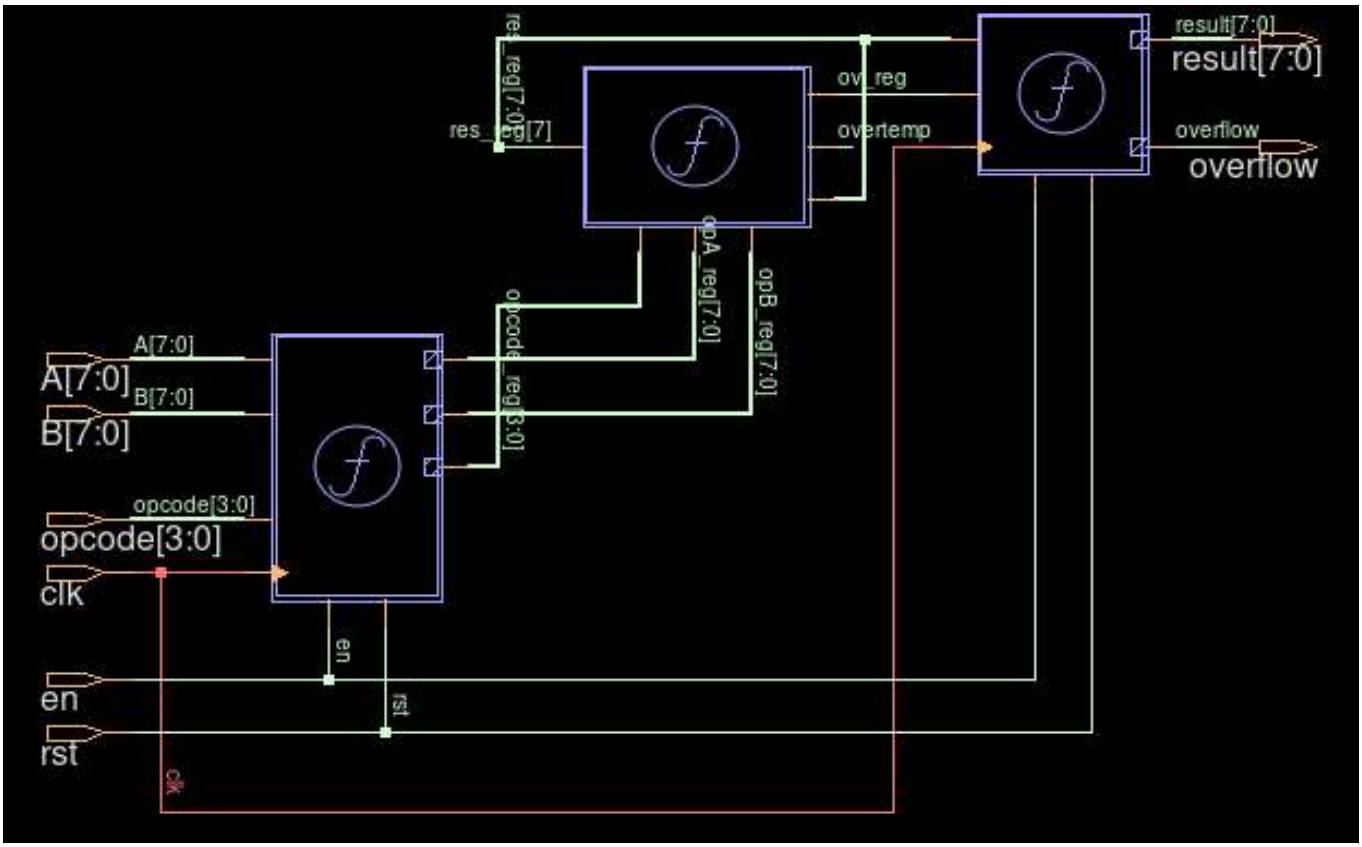
$finish at simulation time 886

V C S S i m u l a t i o n R e p o r t

Schematic Design

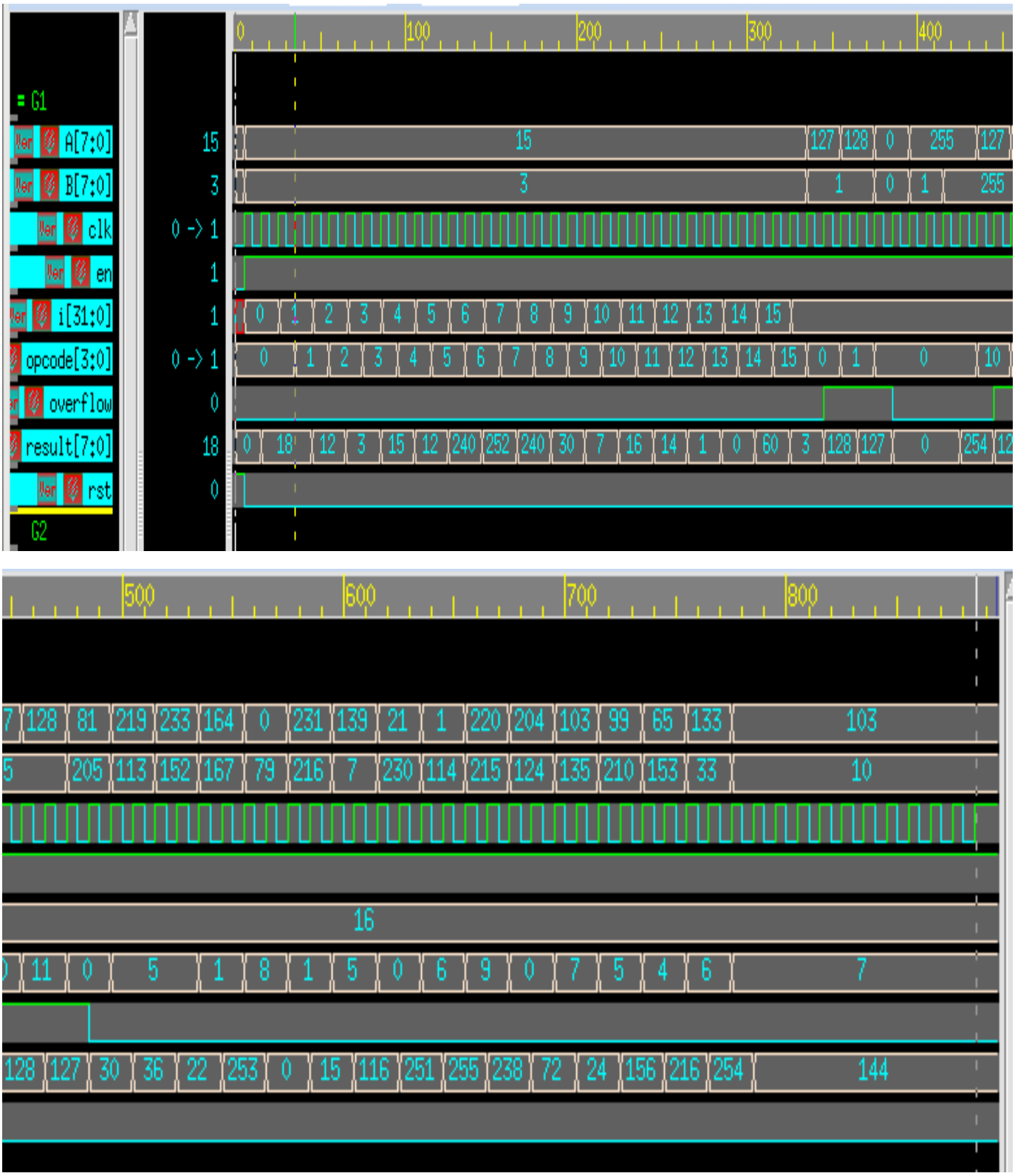


(Schematic View of 8-bit ALU - by VCS and Verdi tool)



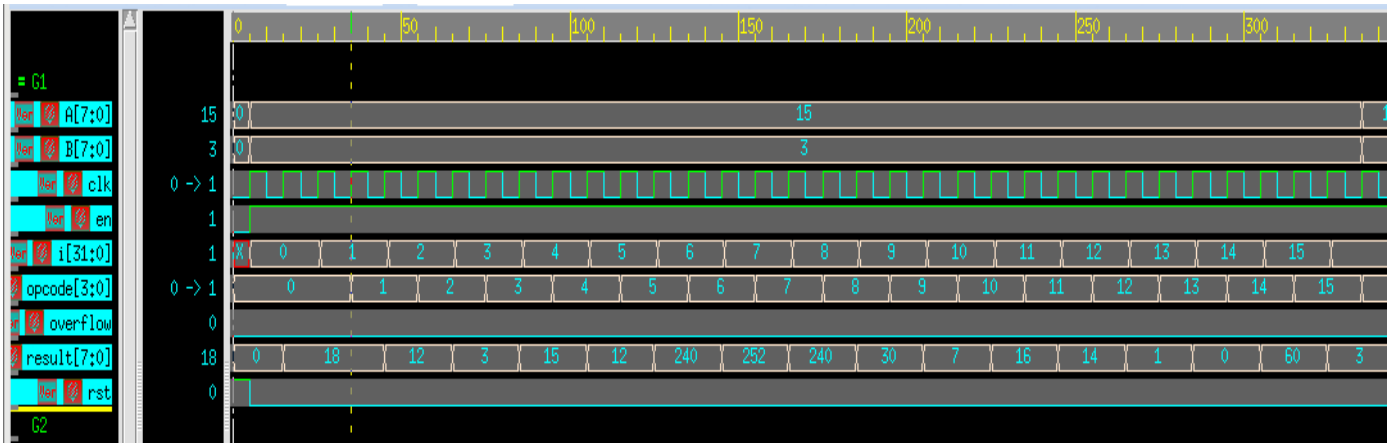
(This is the internal schematic view of synthesizable circuits of 8-bit ALU)

Waveform-



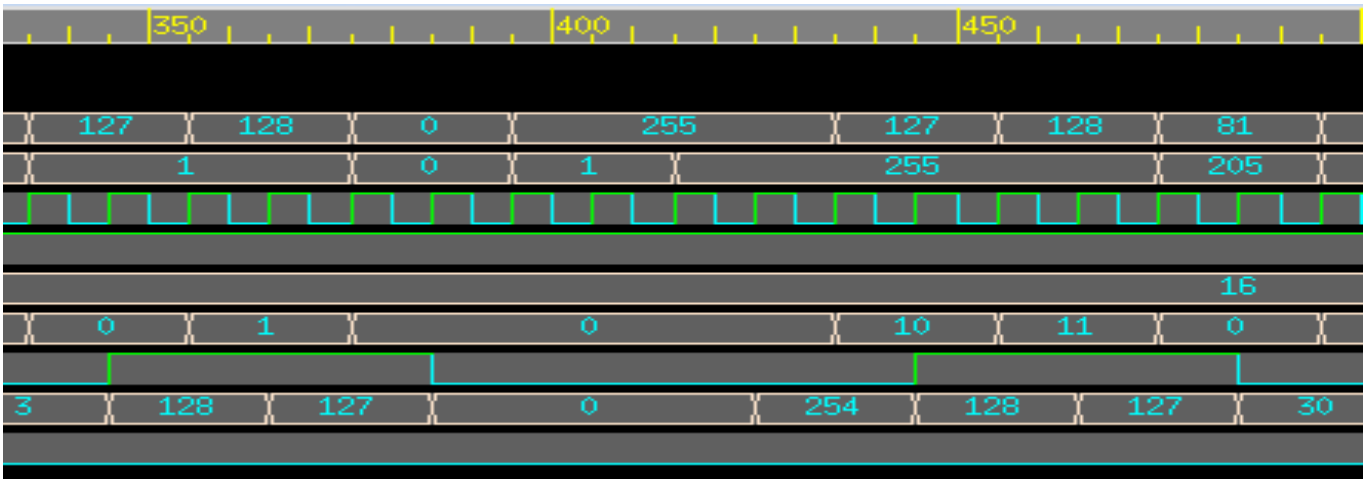
(Output waveform by using the Vcs and Verdi tool)

Test case1: Reset, enable, and all operations Test



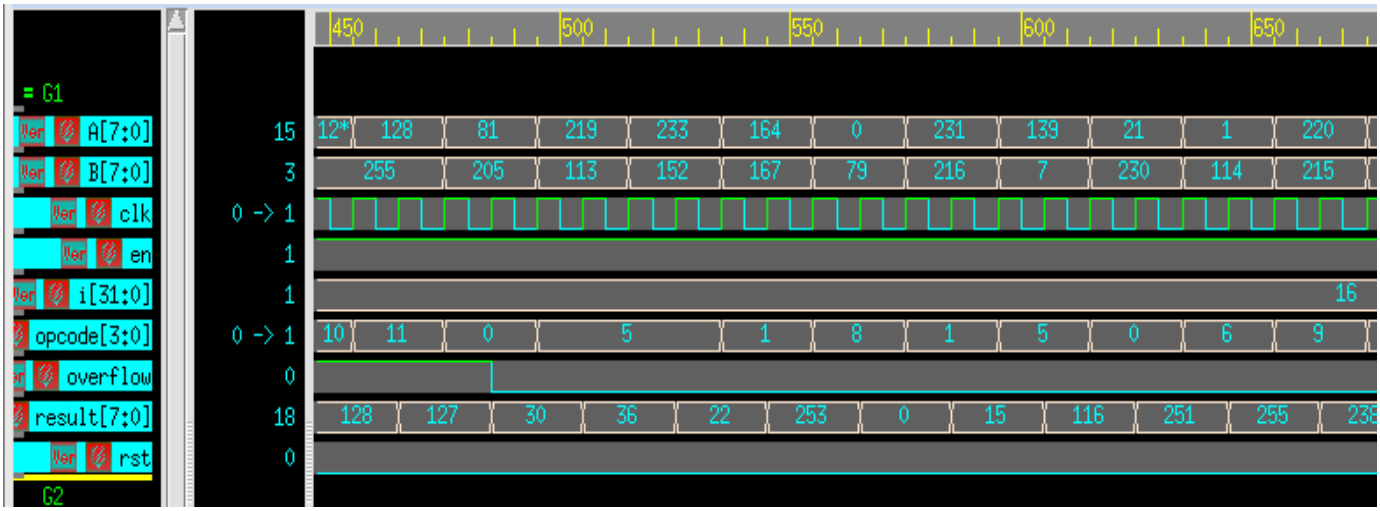
(Test status: checked pass)

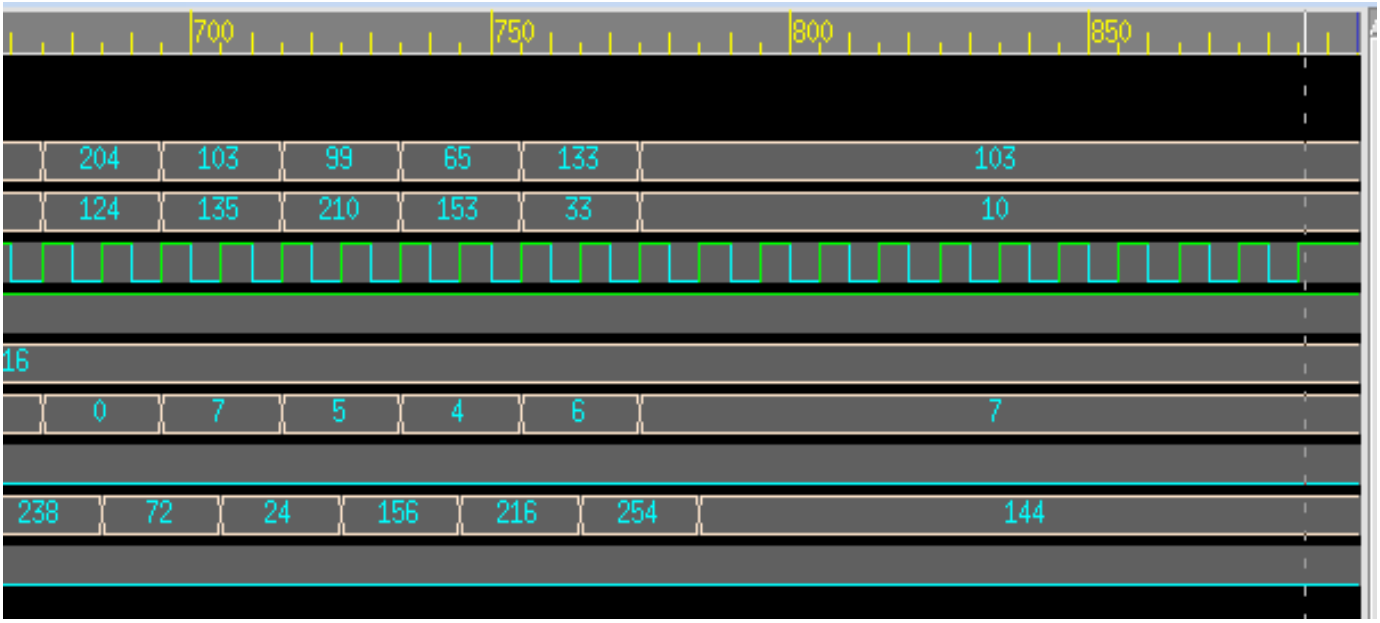
Test case2: Corner cases scenario Tests



(Test status: checked pass)

Test case3: Random data input and opcode TEST-





(Test status: checked pass)

Synthesis: -

Synthesis is the process that converts the RTL code (Verilog/system Verilog) into a gate-level netlists with all the constraints as specified by the designer, which is a list of logic gates, flip-flops, memory and their connections.

Inputs Files for Synthesis: -

1.0 RTL code(.v): Design written in Verilog in behavioural style modelling.
2.0 Logical Library file(.lib): The logic library is also called a timing library or functional library or power library as it contains the functionality, time and power information of standard cells of particular technology.
3.0 Constraint file: It provides design timing, area and power constraints to synthesis tool (mainly clock information).

Physical libraries (Std cell and macros)→extension (.tf) and technology file (.tf)

Synthesis steps:

Synthesis takes place multiple steps:

1**) Elaboration:** Elaboration is the process of expanding your HDL description to represent all instances of all modules (Verilog) into unique objects.

2) Converts RTL into simple logic gates (**Technology independent schematic**).

3) Mapping those gates to actual technology dependent logic gates available in the technology libraries.

4) Optimizing the mapping netlists keeping the constraints set by the designer intact.

Output from Synthesis tool:

1.0 Verilog/mapped netlist (.v)
2.0 Constraint file (.sdc)

Checks performed during the analysis: -

1) Static timing analysis: setup and hold time check
2) Power analysis
3) Area analysis

Netlist format is .V:-

1) It contains logical connectivity of all cells (std cells, Macros).
2) It contains the lists of nets connecting standard cells and macros.
3) Each cell has its own instance/cell and library name.

SDC:-

- Format is .sdc
- These constraints are timing Constraints.
- Theses constraint are mainly used to meet timing requirements of design.

1. Clock definitions: To create different types of clocks
2. Clock uncertainty.
3. Setting input delay
4. Setting output delay
5. Driving Cell
6. Setting loads on output ports
    - Exceptions:
    - Multicycle path
    - False path
    - Half cycle path
    - Disable timing arcs
    - Case analysis

1. Synthesis is the process of converting RTL code (written in Verilog or VHDL) into a gate-level netlist.

2. It bridges the gap between high-level design and physical hardware implementation.

3. The synthesis tool maps RTL constructs to standard cells from a target technology library.

4. It ensures the design meets timing, area, and power constraints.

5. The process includes translation, optimization, and technology mapping stages.

6. Constraints like clock frequency and input/output delays guide the synthesis tool.

7. The output is a netlist consisting of logic gates and flip-flops.

8. Synopsys Design Compiler (DC) is a commonly used synthesis tool.

9. The synthesized netlist is used for gate-level simulation and timing verification.

10. Functional equivalence between RTL and netlist is verified using formal verification.

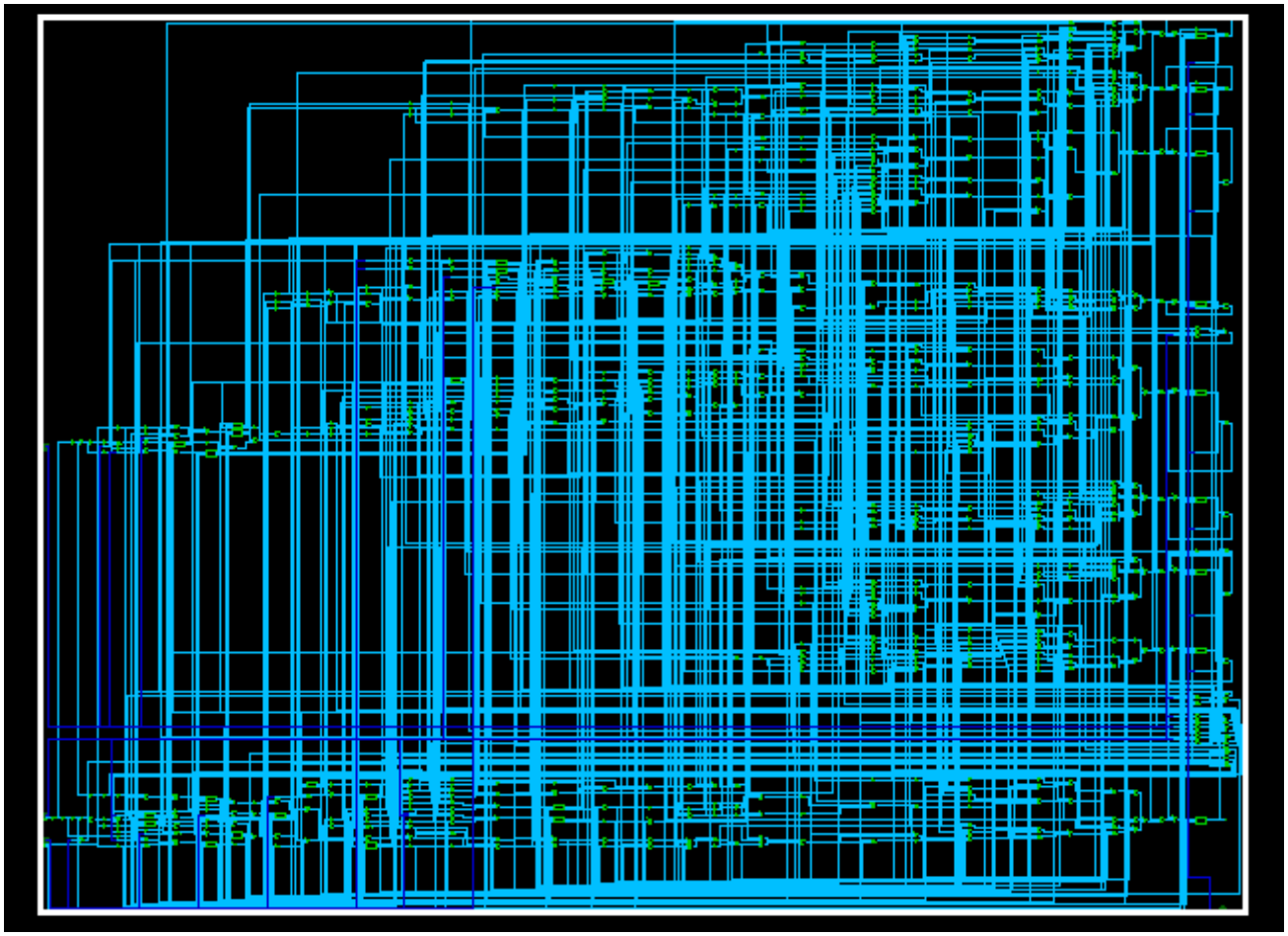
11. After synthesis, reports for timing, area, and power are analyzed.

12. This synthesized design then proceeds to the physical design stage.

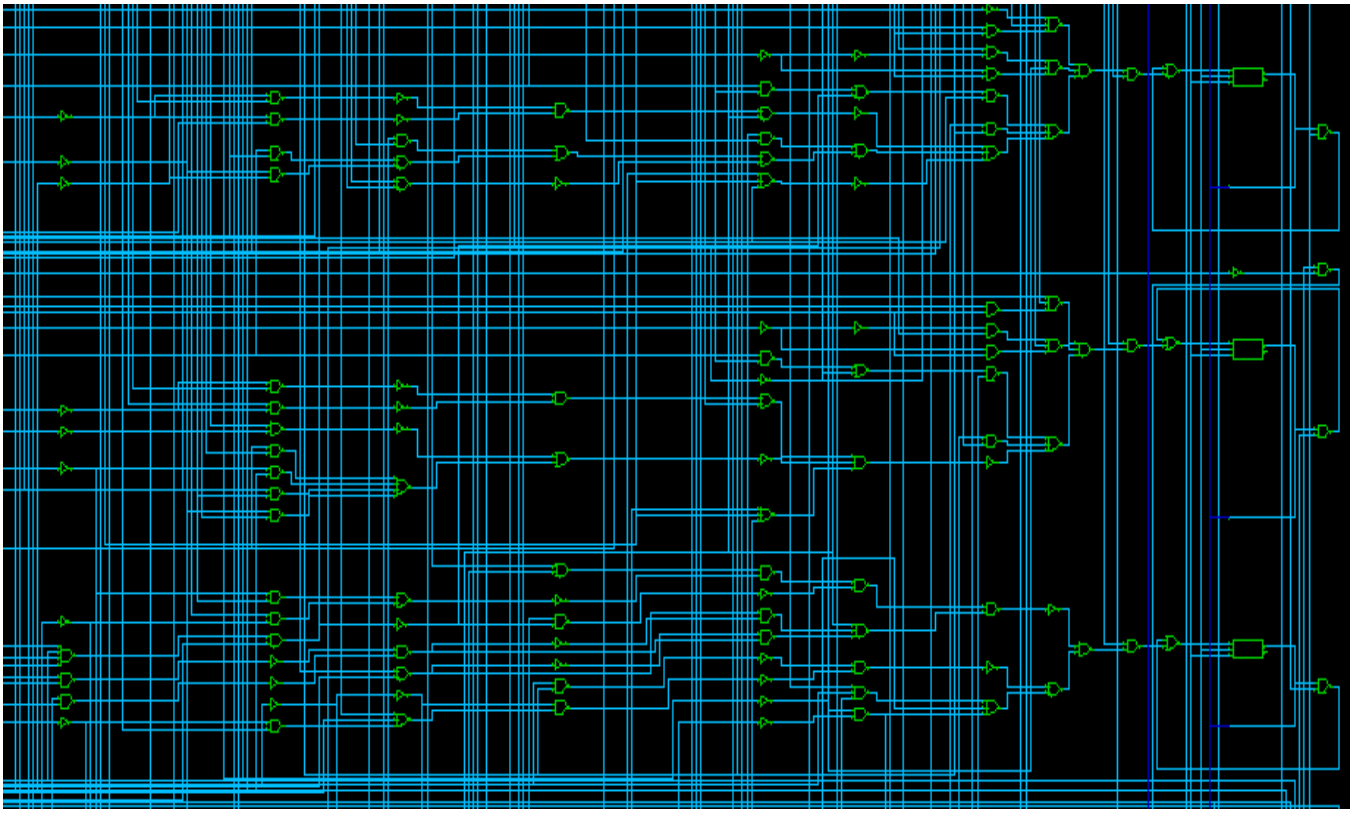
Synthesis by using the SYNOPSYS TOOLS (SDC)-

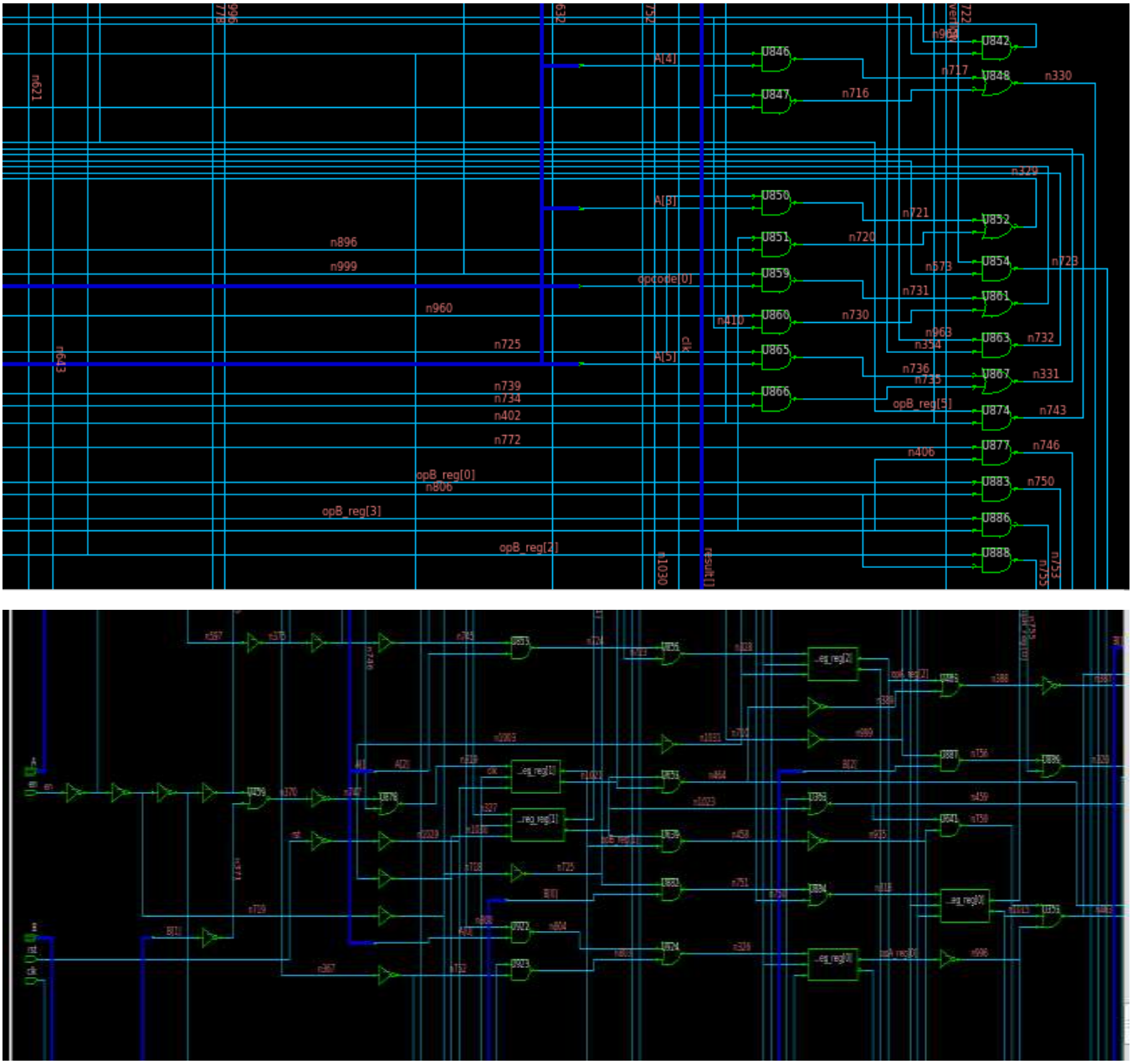


(Schematic View of 8-bit_ALU - by using Synopsys Design Compiler)



(This is the internal schematic view of synthesizable circuits)





(Zoom view inside the ckt -By using the Synopsys Design Compiler (SDC))

Simulation Results

Testbench Design

- The testbench operates synchronously with posedge clock.
- Inputs are applied sequentially for different operation codes.
- Results and overflow flags are printed after each operation.
- The testbench includes both directed test cases (predefined operands) and **overflow** test cases (ex-`A=8'h7F, B=8'h01`).

Verification Scenarios

1. Reset & enable behaviour – verified all outputs reset to zero.

2. Corner scenarios operation – test the initial and boundary cases.

3. Random operations – verified random data input and opcode conditions.

4. 16 ALU operations.

5. Clocked synchronised design with active high reset.

6. Fully synthesizable RTL for FPGA and ASIC implementations.

7. Enable controlled output ensures output updates only when en=1

8. Parametrized and modular design for easy integration in large system.

Simulation and Verification

- **Tools Used:**
- Simulator: Synopsys VCS
- Waveform Debugger: Synopsys Verdi
- Dump File: FSDB format
- Synthesis: Synopsys Design Compiler (DC)
- Target frequency: 715MHz

Results and Analysis

- The ALU successfully performs all arithmetic and logic operations.
- Overflow detection works correctly for boundary conditions.
- Synthesizable and passes Design Compiler checks without timing violations.
- Waveforms verified in Verdi confirm expected operation timing and output transitions.

Key Learnings

- RTL design using synthesizable constructs.
- Overflow and flag handling for arithmetic operations.
- Synchronous testbench development using clock events.
- Waveform debugging and timing verification, area and power analysis report.

Conclusion

The 8-bit ALU project demonstrates the complete RTL design and verification flow — from specification to simulation and synthesis in digital system. This project enhanced

understanding of digital design fundamentals, System Verilog verification, and overflow logic implementation.

Parametrization: Learn how to make the memory modules scalable and adaptable for various applications.

The 8-bit ALU built today forms a miniature brain capable of performing a variety of operations — from addition to bit-level manipulation. Designing this in System Verilog helped me understand how **control signals (opcodes)** steer the functionality of a data-path and how multiple operations can be unified under a single module. It's an essential building block in any processor or embedded system. The results matched the expected truth table, demonstrating correct functionality. By mastering the ALU, I gain insight into how processors execute tasks and handle data. This project not only enhanced my understanding of arithmetic and logical operations but also showed how these concepts come together in complex systems.

It provides a **strong base** for advanced work in processor architecture, UVM verification, and VLSI design automation.

This project demonstrates practical skills in:

- RTL design using System Verilog
- Functional verification using System Verilog testbench
- Debugging using Verdi
- Understanding of ALU operation and synthesis constraints.

Design Testing:

I tested the design by writing the testbench. The output of the testbench shows the design is working correctly as show in the picture. **Finally, our design is functionally correct and passes all simulation cases.** In digital design, ALU is often **called** the **heart of CPU** because it's the component that performs all the fundamental arithmetic, logical, and bitwise operations required to execute program instructions. It is the central processing unit's core component for performing calculations and making decisions.

Advantages-

The ALU (Arithmetic Logic Unit) is one of the most critical components in any processor, microcontroller, or digital system. It performs all arithmetic and logical operations essential for computation and data processing.

- Fast execution of basic arithmetic and logic operations
- Can be parameterized for larger widths
- Clock-driven synchronization needed
- Easily extendable to add new operations

- **Performs Multiple Operations:** An ALU can handle a variety of arithmetic (addition, subtraction, increment, decrement) and logical (AND, OR, XOR, NOT, shift, comparison) operations within a single hardware block. This versatility simplifies the processor design - a single circuit performs multiple functions.
- **High-Speed Data Processing:** Operations in the ALU are performed using hardware-level logic gates, which are much faster than performing the same tasks via software or microcode. Enables real-time computation in processors and embedded systems.
- **Efficient Use of Hardware:** The same circuitry can execute different operations by changing the opcode (operation code), reducing the need for multiple separate units. This saves area, cost, and power on silicon.
- **Supports Parallel and Pipelined Execution:** In advanced designs (like your 8-bit ALU), the structure can be pipelined or replicated to support parallel computation, improving throughput. This makes it scalable for higher-bit processors (16-bit, 32-bit, etc.).
- **Essential for Decision-Making:** The logical operations of the ALU (like comparisons) are used for branching, condition checking, and control flow in processors. It enables CPUs to make decisions based on data values.
- **Basis of All Computation:** Every mathematical or logical function in computing - from simple addition to complex algorithm execution -ultimately depends on ALU operations. It's the core building block of CPUs, DSPs, and FPGAs.
- **Easily Scalable and Configurable:** ALUs can be designed as parameterized modules (like yours, with `DATA_WIDTH` = 8). This makes it easy to reuse and scale the design for 16-bit, 32-bit, or even 64-bit architectures.
- **Improves System Performance:** A well-optimized ALU with low latency and good timing closure contributes directly to overall processor speed and system efficiency.

In my 8-bit ALU project, the advantages are: -

- I achieved multi-functionality (15+ operations in one design).
- The design is parameterized, synthesizable, and verified.
- It can be used as a core module in an embedded processor or FPGA-based system.

Application-

An Arithmetic Logic Unit (ALU) is the core computational unit inside a processor (CPU, GPU, DSP, or microcontroller). It performs arithmetic (like addition, subtraction, shifting) and logical (like AND, OR, XOR) operations on binary data.

Main Applications of an ALU: -

Central Processing Unit (CPU).

- ALU is the heart of the CPU.
- It performs mathematical and logical operations required by instructions executed by the processor.
- Every instruction that involves computation (like `ADD`, `SUB`, `AND`, `CMP`) is executed by the ALU.

Example:

When executing `ADD R1, R2, R3`, the CPU's control unit instructs the ALU to add the contents of R2 and R3 and store the result in R1.

Digital Signal Processing (DSP)

- Used in DSP processors for fast arithmetic operations such as multiplication, addition, and accumulation. Essential in audio/video processing, image filtering, communications, and control systems.

Microcontrollers and Embedded Systems

- ALU handles arithmetic and logic in small embedded processors (e.g., in washing machines, cars, IoT devices). Used in real-time decision making, sensor data computation, and control algorithms.

Cryptographic and Security Systems

- ALU operations are used in encryption/decryption, hashing, and modular arithmetic for secure data processing.

Floating Point and Integer Units

- In modern processors, ALUs are part of execution units:
- Integer ALU → addition, subtraction, logic ops
- Floating-Point ALU (FPU) → handles real-number arithmetic

Control Units and Automation Systems

- ALUs are used for decision-making logic in controllers, PLCs, and industrial automation for comparing, adding sensor data, or adjusting system outputs.

Custom VLSI or ASIC Design

- In SoC (System-on-Chip) or ASIC design, ALUs are designed, verified, and synthesized as part of data path blocks.
- Verification engineers write System Verilog testbenches to check arithmetic correctness, overflow handling, and flag generation.

In my Resume-

- ✓ Designed and verified an 8-bit Arithmetic Logic Unit (ALU) using System Verilog; performed functional verification, synthesis, and timing analysis. The ALU supports arithmetic, logic, and shift operations used in processor datapaths and embedded systems.

The **main application** of an Arithmetic Logic Unit (ALU) is to perform arithmetic and logical operations on binary numbers within a computer's Central Processing Unit (CPU).

Core applications in computing:

- ✓ **Arithmetic operations**: Performs basic calculations like addition and subtraction, and can handle multiplication.
- ✓ **Logical operations**: Executes bitwise logical functions such as AND, OR, NOT, and XOR.
- ✓ **Data comparison**: Compares values to determine relationships, such as greater than or less than.
- ✓ **Data manipulation**: Handles operations like incrementing and decrementing values by one.
- ✓ **Bit shifting**: Moves bits to the left or right, which is a common operation in data processing.

Specialized and modern applications:

- ✓ **Internet of Things (IoT)**: Researchers are designing specialized, power-efficient ALU architectures to meet the low-power demands of IoT devices.
- ✓ **Application-specific integrated circuits (ASICs)**: ALUs are customized for specific applications to reduce power consumption and improve efficiency for a particular task.
- ✓ **Embedded systems**: Their role is critical in the processor designs of many embedded systems, where they handle a wide range of data processing needs.

- Central Processing Units (CPUs)
- Embedded controllers
- Signal processing systems
- Hardware arithmetic blocks
- Digital calculators

Simulation Results

Report design Area

Report_timing

dc_shell> report_timing -to A[6] -significant_digits 6

```
****************************************
Report : timing
        -path full
        -delay max
        -max_paths 1
Design : ALU_8_bit
Version: V-2023.12-SP4
Date   : Mon Oct 20 11:20:38 2025
****************************************
Operating Conditions: ff1p16v125c   Library: saed32rvt_ff1p16v125c
Wire Load Model Mode: enclosed
  Startpoint: A[6] (input port clocked by clk)
  Endpoint: opA_reg_reg[6]
            (rising edge-triggered flip-flop clocked by clk)
  Path Group: clk
  Path Type: max
  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  ALU_8_bit          8000                  saed32rvt_ff1p16v125c
  Point                                 Incr       Path
  -----------------------------------------------------------
  clock clk (rise edge)                 0.000000   0.000000
  clock network delay (ideal)           0.470000   0.470000
  input external delay                  1.000000   1.470000 f
  A[6] (in)                             0.000000   1.470000 f
  U925/Y (AND2X1_RVT)                   0.177057   1.647057 f
  U540/Y (OR2X1_RVT)                    0.028633   1.675690 f
```

opA_reg_reg[6]/D (DFFARX1_RVT) 0.010957 1.686648 f

data arrival time 1.686648

clock clk (rise edge) 1.400000 1.400000

clock network delay (ideal) 0.470000 1.870000

clock uncertainty -0.070000 1.800000

opA_reg_reg[6]/CLK (DFFARX1_RVT) 0.000000 1.800000 r

library setup time -0.025588 1.774412

data required time 1.774412

---

data required time 1.774412

data arrival time -1.686648

---

slack (MET) 0.087765

1

Finally, we obtained the **slack value** is **greater than 0.**

Hold report-

dc_shell> report_timing -significant_digits 6 -delay_type min

****

Report : timing

-path full

-delay min

-max_paths 1

Design : ALU_8_bit

Version: V-2023.12-SP4

Date : Mon Oct 20 11:36:41 2025

****

Operating Conditions: ff1p16v125c Library: saed32rvt_ff1p16v125c

```
Wire Load Model Mode: enclosed
  Startpoint: result_reg[7]
              (rising edge-triggered flip-flop clocked by clk)
  Endpoint: result_reg[7]
            (rising edge-triggered flip-flop clocked by clk)
  Path Group: clk
  Path Type: min
  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  ALU_8_bit          8000                  saed32rvt_ff1p16v125c
  Point                                 Incr       Path
  ------------------------------------------------------------
  clock clk (rise edge)                 0.000000   0.000000
  clock network delay (ideal)           0.000000   0.000000
  result_reg[7]/CLK (DFFARX1_RVT)       0.000000   0.000000 r
  result_reg[7]/Q (DFFARX1_RVT)         0.061613   0.061613 r
  U819/Y (AND2X1_RVT)                   0.079888   0.141502 r
  U356/Y (OR2X1_RVT)                    0.032005   0.173506 r
  result_reg[7]/D (DFFARX1_RVT)         0.009510   0.183016 r
  data arrival time                                0.183016
  clock clk (rise edge)                 0.000000   0.000000
  clock network delay (ideal)           0.000000   0.000000
  result_reg[7]/CLK (DFFARX1_RVT)       0.000000   0.000000 r
  library hold time                     0.002580   0.002580
  data required time                               0.002580
  ------------------------------------------------------------
  data required time                               0.002580
```

data arrival time -0.183016

---

slack (MET) 0.180437

1

Report_qor

dc_shell> report_qor

****

Report : qor

Design : ALU_8_bit

Version: V-2023.12-SP4

Date : Mon Oct 20 11:39:02 2025

****

Timing Path Group 'clk'

Levels of Logic: 27.00

Critical Path Length: 1.69

Critical Path Slack: -0.38

Critical Path Clk Period: 1.40

Total Negative Slack: -9.86

No. of Violating Paths: 29.00

Worst Hold Violation: 0.00

Total Hold Violation: 0.00

No. of Hold Violations: 0.00

Cell Count

Hierarchical Cell Count: 0

Hierarchical Port Count: 0

Leaf Cell Count: 720

Buf/Inv Cell Count: 243

Buf Cell Count: 67

Inv Cell Count: 176

CT Buf/Inv Cell Count: 0

Combinational Cell Count: 691

Sequential Cell Count: 29

Macro Count: 0

Area

Combinational Area: 1301.979762

Noncombinational Area: 206.873219

Buf/Inv Area: 360.630344

Total Buffer Area: 136.22

Total Inverter Area: 224.41

Macro/Black Box Area: 0.000000

Net Area: 227.124558

Cell Area: 1508.852981

Design Area: 1735.977539

Design Rules

Total Number of Nets: 763

Nets With Violations: 0

Max Trans Violations: 0

Max Cap Violations: 0

Hostname: prac3.trg.vlsiexpert.in

Compile CPU Statistics

Resource Sharing: 0.07

Logic Optimization: 0.26

Mapping Optimization: 5.15

Overall Compile Time: 8.39

Overall Compile Wall Clock Time: 8.93

Design WNS: 0.38 TNS: 9.86 Number of Violating Paths: 29

Design (Hold) WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

1

Report_area:

dc_shell> report_area

****

Report : area

Design : ALU_8_bit

Version: V-2023.12-SP4

Date : Mon Oct 20 11:42:21 2025

****

Library(s) Used:

saed32rvt_ff1p16v125c (File: /ve/po_home/b23n_sarif_727/internship_mux/ref/lib/stdcell_rvt/saed32rvt_ff1p16v125c.db)

Number of ports: 32

Number of nets: 763

Number of cells: 720

Number of combinational cells: 691

Number of sequential cells: 29

Number of macros/black boxes: 0

Number of buf/inv: 243

Number of references: 13

Combinational area: 1301.979762

Buf/Inv area: 360.630344

Noncombinational area: 206.873219

Macro/Black Box area: 0.000000

Net Interconnect area: 227.124558

Total cell area: 1508.852981

Total area: 1735.977539

1

Report_power:

dc_shell> report_power

****

Report : power

-analysis_effort low

Design : ALU_8_bit

Version: V-2023.12-SP4

Date : Mon Oct 20 11:43:35 2025

****

Library(s) Used:

saed32rvt_ff1p16v125c (File: /ve/po_home/b23n_sarif_727/internship_mux/ref/lib/stdcell_rvt/saed32rvt_ff1p16v125c.db)

Operating Conditions: ff1p16v125c Library: saed32rvt_ff1p16v125c

Wire Load Model Mode: enclosed

Design Wire Load Model Library

ALU_8_bit 8000 saed32rvt_ff1p16v125c

Global Operating Voltage = 1.16

Power-specific unit information :

Voltage Units = 1V

Capacitance Units = 1.000000ff

Time Units = 1ns

Dynamic Power Units = 1uW (derived from V,C,T units)

Leakage Power Units = 1pW

Attributes

---

i - Including register clock pin internal power

Cell Internal Power = 919.4045 uW (98%)

Net Switching Power = 19.6392 uW (2%)

Total Dynamic Power = 939.0438 uW (100%)

Cell Leakage Power = 12.0990 mW

Internal Switching Leakage Total

Power Group Power Power Power Power (%) Attrs

---

io_pad 0.0000 0.0000 0.0000 0.0000- (0.00%)

memory 0.0000 0.0000 0.0000 0.0000- (0.00%)

black_box 0.0000 0.0000 0.0000 0.0000- (0.00%)

clock_network 949.8910 0.0000 0.0000 949.8910- (7.29%) i

register -1.5872e+02 2.3443 2.2981e+09 2.1417e+03 (16.43%)

sequential 0.0000 0.0000 0.0000 0.0000- (0.00%)

combinational 128.2324 17.2949 9.8008e+09 9.9464e+03- (76.29%)

Total 919.4044 uW 19.6392 uW 1.2099e+10 pW 1.3038e+04 uW

1

Constraints File: - ALU_8_bit.sdc

```
create_clock -period 1.4 [get_ports clk]
set_clock_uncertainty -setup 0.07 [get_clocks clk]
set_clock_latency -source -max 0.0700 [get_clocks clk]
set_clock_latency -max 0.400 [get_clocks clk]
set_clock_transition -max 0.25 [get_clocks clk]

set_input_delay -max 1 -clock clk [all_inputs]
set_input_transition -max 1 [all_inputs]

set_output_delay -max 1 -clock clk [all_outputs]
set_load -max 1 [all_outputs]
```

Compiler_ultra

dc_shell> compile_ultra

RM-Info: Completed script /ve/po_home/b23n_sarif_727/internship_mux/DC/rm_setup/dc_setup.tcl

Loading db file '/ve/po_home/b23n_sarif_727/internship_mux/ref/lib/stdcell_rvt/saed32rvt_ff1p16v125c.db'

Loading db file '/ve/po_home/b23n_sarif_727/internship_mux/ref/lib/stdcell_rvt/saed32rvt_tt0p78vn40c.db'

Loading db file '/ve/po_home/b23n_sarif_727/internship_mux/ref/lib/stdcell_rvt/saed32rvt_ss0p7vn40c.db'

Loading db file '/usr/synopsys/syn/V-2023.12-SP4/libraries/syn/dw_foundation.sldb'

Running PRESTO HDLC

Compiling source file ./../simulation/alu_8_bit.sv

Presto compilation completed successfully.

Loading db file '/usr/synopsys/syn/V-2023.12-SP4/libraries/syn/**gtech.db**'

Loading db file '/usr/synopsys/syn/V-2023.12-SP4/libraries/syn/**standard.sldb**'

Loading link library 'gtech'

Running PRESTO HDLC

Warning: ./../simulation/alu_8_bit.sv:59: signed to unsigned conversion occurs. (VER-318)

Warning: ./../simulation/alu_8_bit.sv:63: signed to unsigned conversion occurs. (VER-318)

Warning: ./../simulation/alu_8_bit.sv:65: unsigned to signed assignment occurs. (VER-318)

Warning: ./../simulation/alu_8_bit.sv:66: unsigned to signed assignment occurs. (VER-318)

Warning: ./../simulation/alu_8_bit.sv:69: unsigned to signed assignment occurs. (VER-318)

Warning: ./../simulation/alu_8_bit.sv:69: DEFAULT branch of CASE statement cannot be reached. (ELAB-311)

Statistics for case statements in always block at line 30 in file

'./../simulation/alu_8_bit.sv'

```
=============================================
|         Line          |  full/ parallel  |
=============================================
|          35           |    user/user     |
=============================================
```

Inferred memory devices in process

in routine ALU_8_bit line 16 in file

'./../simulation/alu_8_bit.sv'.

```
=============================================================================
===============
|   Register Name    |   Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
=============================================================================
===============
|  opcode_reg_reg    | Flip-flop |   4   |  Y  | N  | Y  | N  | N  | N  | N  |
|    opA_reg_reg     | Flip-flop |   8   |  Y  | N  | Y  | N  | N  | N  | N  |
|    opB_reg_reg     | Flip-flop |   8   |  Y  | N  | Y  | N  | N  | N  | N  |
=============================================================================
===============
```

Inferred memory devices in process

in routine ALU_8_bit line 73 in file

'./../simulation/alu_8_bit.sv'.

```
=============================================================================
===============
|   Register Name    |   Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
=============================================================================
===============
|   overflow_reg     | Flip-flop |   1   |  N  | N  | Y  | N  | N  | N  | N  |
|    result_reg      | Flip-flop |   8   |  Y  | N  | Y  | N  | N  | N  | N  |
```

```
============================================================
==============
Presto compilation completed successfully. (ALU_8_bit)
Elaborated 1 design.
Current design is now 'ALU_8_bit'.
Current design is 'ALU_8_bit'.

Reading SDC version 2.1...
Information: Performing leakage power optimization. (PWR-850)
Alib files are up-to-date.
Information: Evaluating DesignWare library utilization. (UISN-27)
============================================================
============
| DesignWare Building Block Library  |         Version         | Available |
============================================================
============
| Basic DW Building Blocks           | V-2023.12-DWBB_202312.4 |     *     |
| Licensed DW Building Blocks        | V-2023.12-DWBB_202312.4 |     *     |
============================================================
============

============================================================
====================================
| Flow Information                                           |
------------------------------------------------------------------------------------------
| Flow         | Design Compiler WLM                                  |
| Command Line | compile_ultra                                        |
============================================================
====================================
| Design Information                    | Value                    |
```

```
================================================================
====================================
| Number of Scenarios                  | 0                     |
| Leaf Cell Count                      | 188                   |
| Number of User Hierarchies           | 0                     |
| Sequential Cell Count                | 29                    |
| Macro Count                          | 0                     |
| Number of Power Domains              | 0                     |
| Number of Path Groups                | 2                     |
| Number of VT Class                   | 1                     |
| Number of Clocks                     | 1                     |
| Number of Dont Touch Cells           | 74                    |
| Number of Dont Touch Nets            | 0                     |
| Number of Size Only Cells            | 0                     |
| Design with UPF Data                 | false                 |
================================================================
```

Mapping netlist file (ALU_8_bit.mapped.v) created by tool-

```
////////////////////////////////////////////////////////////
// Created by: Synopsys DC Ultra(TM) in wire load mode
// Version   : V-2023.12-SP4
// Date      : Mon Oct 20 11:19:32 2025
////////////////////////////////////////////////////////////
module ALU_8_bit ( clk, rst, en, A, B, opcode, result, overflow );
  input [7:0] A;
  input [7:0] B;
  input [3:0] opcode;
  output [7:0] result;
```

```
input clk, rst, en;
output overflow;
wire   n309, n310, n311, n312, n313, n314, n315, n316, n317, n318, n319,
       n320, n321, n322, n323, n324, n325, n326, n327, n328, n329, n330,
       n331, n332, n333, n334, n335, n336, n337, n341, n342, n343, n344,
       n345, n346, n348, n349, n351, n352, n353, n354, n355, n356, n357,
       n358, n359, n360, n361, n362, n363, n364, n365, n366, n367, n368,
       n369, n370, n371, n372, n373, n374, n375, n376, n377, n378, n379,
       n380, n381, n382, n383, n384, n385, n386, n387, n388, n389, n390,
       n391, n392, n393, n394, n395, n396, n397, n398, n399, n400, n401,
       n402, n403, n404, n405, n406, n407, n408, n410, n411, n412, n413,
       n414, n415, n416, n417, n418, n419, n420, n421, n422, n423, n424,
       n425, n426, n427, n429, n430, n431, n432, n433, n434, n435, n436,
       n437, n438, n439, n440, n441, n442, n443, n444, n445, n446, n447,
       n448, n449, n450, n451, n452, n453, n454, n455, n456, n457, n458,
       n459, n460, n461, n462, n463, n464, n465, n466, n467, n468, n469,
       n470, n471, n472, n473, n474, n475, n476, n477, n478, n479, n480,
       n481, n482, n483, n484, n485, n486, n487, n488, n489, n490, n491,
       n492, n493, n494, n495, n496, n497, n498, n499, n500, n501, n502,
       n503, n504, n505, n506, n507, n508, n509, n510, n511, n512, n513,
       n514, n515, n516, n517, n518, n519, n520, n521, n522, n523, n524,
       n525, n526, n527, n528, n529, n530, n531, n532, n533, n534, n535,
       n536, n537, n538, n539, n540, n541, n542, n543, n544, n545, n546,
       n547, n548, n549, n550, n551, n552, n553, n554, n555, n556, n557,
       n558, n559, n560, n561, n562, n563, n564, n565, n566, n567, n568,
       n569, n570, n571, n572, n573, n574, n575, n576, n577, n578, n579,
       n580, n581, n582, n583, n584, n585, n586, n587, n588, n589, n590,
```

n591, n592, n593, n594, n595, n596, n597, n598, n599, n600, n601,
n602, n603, n604, n605, n606, n607, n608, n609, n610, n611, n612,
n613, n614, n615, n616, n617, n618, n619, n620, n621, n622, n623,
n624, n625, n626, n627, n628, n629, n630, n631, n632, n633, n634,
n635, n636, n637, n638, n639, n640, n641, n642, n643, n644, n645,
n646, n647, n648, n649, n650, n651, n652, n653, n654, n655, n656,
n657, n658, n659, n660, n661, n662, n663, n664, n665, n666, n667,
n668, n669, n670, n671, n672, n673, n674, n675, n676, n677, n678,
n679, n680, n681, n682, n683, n684, n685, n686, n687, n688, n689,
n690, n691, n692, n693, n694, n695, n696, n697, n698, n699, n700,
n701, n702, n703, n704, n705, n706, n707, n708, n709, n710, n711,
n712, n713, n714, n715, n716, n717, n718, n719, n720, n721, n722,
n723, n724, n725, n726, n727, n728, n729, n730, n731, n732, n733,
n734, n735, n736, n737, n738, n739, n740, n741, n742, n743, n744,
n745, n746, n747, n748, n749, n750, n751, n752, n753, n754, n755,
n756, n757, n758, n759, n760, n761, n762, n763, n764, n765, n766,
n767, n768, n769, n770, n771, n772, n773, n774, n775, n776, n777,
n778, n779, n780, n781, n782, n783, n784, n785, n786, n787, n788,
n789, n790, n791, n792, n793, n794, n795, n796, n797, n798, n799,
n800, n801, n802, n803, n804, n805, n806, n807, n808, n809, n810,
n811, n812, n813, n814, n815, n816, n817, n818, n819, n820, n821,
n822, n823, n824, n825, n826, n827, n828, n829, n830, n831, n832,
n833, n834, n835, n836, n837, n838, n839, n840, n841, n842, n843,
n844, n845, n846, n847, n848, n849, n850, n851, n852, n853, n854,
n855, n856, n857, n858, n859, n860, n861, n862, n863, n864, n865,
n866, n867, n868, n869, n870, n871, n872, n873, n874, n875, n876,
n877, n878, n879, n880, n881, n882, n883, n884, n885, n886, n887,

```
    n888, n889, n890, n891, n892, n893, n894, n895, n896, n897, n898,
    n899, n900, n901, n902, n903, n904, n905, n906, n907, n908, n909,
    n910, n911, n912, n913, n914, n915, n916, n917, n918, n919, n920,
    n921, n922, n923, n924, n925, n926, n927, n928, n929, n930, n931,
    n932, n933, n934, n935, n936, n937, n938, n939, n940, n941, n942,
    n943, n944, n945, n946, n947, n948, n949, n950, n951, n952, n953,
    n954, n955, n956, n957, n958, n959, n960, n961, n962, n963, n964,
    n965, n966, n967, n968, n969, n970, n971, n972, n973, n974, n975,
    n976, n977, n978, n979, n980, n981, n982, n983, n984, n985, n986,
    n987, n988, n989, n990, n991, n992, n993, n994, n995, n996, n997,
    n998, n999, n1000, n1001, n1002, n1003, n1004, n1005, n1006, n1007,
    n1008, n1009, n1010, n1011, n1012, n1013, n1014, n1015, n1016, n1017,
    n1018, n1019, n1020, n1021, n1022, n1023, n1024, n1025, n1026, n1027,
    n1028, n1029, n1030, n1031;
wire   [7:0] opA_reg;
wire   [7:0] opB_reg;
wire   [3:0] opcode_reg;

DFFARX1_RVT \opcode_reg_reg[1]  ( .D(n335), .CLK(clk), .RSTB(n1029), .Q(
    opcode_reg[1]), .QN(n1022) );
DFFARX1_RVT \opcode_reg_reg[0]  ( .D(n334), .CLK(clk), .RSTB(n1031), .Q(
    n1011), .QN(n1016) );
DFFARX1_RVT \opA_reg_reg[5]  ( .D(n331), .CLK(clk), .RSTB(n1031), .Q(n1026),
    .QN(n1018) );
DFFARX1_RVT \opA_reg_reg[4]  ( .D(n330), .CLK(clk), .RSTB(n1031), .Q(n1025),
    .QN(n1017) );
DFFARX1_RVT \opA_reg_reg[2]  ( .D(n328), .CLK(clk), .RSTB(n1031), .Q(
```

```
    opA_reg[2]), .QN(n1004) );
DFFARX1_RVT overflow_reg ( .D(n317), .CLK(clk), .RSTB(n1029), .Q(overflow)
     );
DFFARX1_RVT \result_reg[7]  ( .D(n316), .CLK(clk), .RSTB(n1030), .Q(
    result[7]) );
DFFARX1_RVT \result_reg[6]  ( .D(n315), .CLK(clk), .RSTB(n1030), .Q(
    result[6]) );
DFFARX1_RVT \result_reg[5]  ( .D(n314), .CLK(clk), .RSTB(n1030), .Q(
    result[5]) );
DFFARX1_RVT \result_reg[4]  ( .D(n313), .CLK(clk), .RSTB(n1030), .Q(
    result[4]) );
DFFARX1_RVT \result_reg[3]  ( .D(n312), .CLK(clk), .RSTB(n1030), .Q(
    result[3]) );
DFFARX1_RVT \result_reg[2]  ( .D(n311), .CLK(clk), .RSTB(n1030), .Q(
    result[2]) );
DFFARX1_RVT \result_reg[1]  ( .D(n310), .CLK(clk), .RSTB(n1030), .Q(
    result[1]) );
DFFARX1_RVT \result_reg[0]  ( .D(n309), .CLK(clk), .RSTB(n1030), .Q(
    result[0]) );
DFFARX1_RVT \opB_reg_reg[1]  ( .D(n319), .CLK(clk), .RSTB(n1029), .Q(
    opB_reg[1]), .QN(n1021) );
DFFARX1_RVT \opB_reg_reg[0]  ( .D(n318), .CLK(clk), .RSTB(n1029), .Q(
    opB_reg[0]), .QN(n1015) );
DFFARX1_RVT \opB_reg_reg[6]  ( .D(n324), .CLK(clk), .RSTB(n1029), .Q(
    opB_reg[6]), .QN(n1014) );
DFFARX1_RVT \opB_reg_reg[5]  ( .D(n323), .CLK(clk), .RSTB(n1029), .Q(
    opB_reg[5]), .QN(n1013) );
```

```
DFFARX1_RVT \opB_reg_reg[4]  ( .D(n322), .CLK(clk), .RSTB(n1029), .Q(
      opB_reg[4]), .QN(n1012) );
DFFARX1_RVT \opB_reg_reg[3]  ( .D(n321), .CLK(clk), .RSTB(n1029), .Q(
      opB_reg[3]), .QN(n1010) );
DFFARX1_RVT \opB_reg_reg[7]  ( .D(n325), .CLK(clk), .RSTB(n1029), .Q(
      opB_reg[7]), .QN(n1009) );
DFFARX1_RVT \opB_reg_reg[2]  ( .D(n320), .CLK(clk), .RSTB(n1029), .Q(
      opB_reg[2]), .QN(n1007) );
DFFARX1_RVT \opA_reg_reg[6]  ( .D(n332), .CLK(clk), .RSTB(n1030), .Q(n1027),
      .QN(n1019) );
DFFARX1_RVT \opcode_reg_reg[3]  ( .D(n337), .CLK(clk), .RSTB(n1029), .Q(
      opcode_reg[3]), .QN(n1024) );
DFFARX2_RVT \opA_reg_reg[1]  ( .D(n327), .CLK(clk), .RSTB(n1030), .Q(
      opA_reg[1]), .QN(n1023) );
DFFARX1_RVT \opcode_reg_reg[2]  ( .D(n336), .CLK(clk), .RSTB(n1029), .Q(
      opcode_reg[2]), .QN(n1006) );
DFFARX1_RVT \opA_reg_reg[0]  ( .D(n326), .CLK(clk), .RSTB(n1030), .Q(
      opA_reg[0]), .QN(n1005) );
DFFARX1_RVT \opA_reg_reg[3]  ( .D(n329), .CLK(clk), .RSTB(n1031), .Q(
      opA_reg[3]), .QN(n1008) );
DFFARX1_RVT \opA_reg_reg[7]  ( .D(n333), .CLK(clk), .RSTB(n1030), .Q(n1028),
      .QN(n1020) );
AND3X1_RVT U343 ( .A1(n345), .A2(n818), .A3(n636), .Y(n672) );
OR2X1_RVT U344 ( .A1(n816), .A2(n920), .Y(n817) );
AND4X1_RVT U345 ( .A1(n727), .A2(n874), .A3(n604), .A4(n1009), .Y(n631) );
OR2X1_RVT U346 ( .A1(n604), .A2(n927), .Y(n607) );
AND2X1_RVT U347 ( .A1(n345), .A2(n433), .Y(n432) );
```

```
INVX0_RVT U348 ( .A(n343), .Y(n427) );
INVX0_RVT U349 ( .A(n544), .Y(n691) );
OR2X2_RVT U350 ( .A1(n434), .A2(n612), .Y(n615) );
INVX4_RVT U351 ( .A(n529), .Y(n782) );
INVX1_RVT U352 ( .A(n625), .Y(n760) );
OR3X1_RVT U353 ( .A1(n759), .A2(n1015), .A3(n996), .Y(n463) );
OR2X1_RVT U354 ( .A1(n518), .A2(n385), .Y(n341) );
AND2X1_RVT U355 ( .A1(n349), .A2(n341), .Y(n348) );
OR2X1_RVT U356 ( .A1(n706), .A2(n705), .Y(n316) );
OR2X1_RVT U357 ( .A1(n404), .A2(n689), .Y(n342) );
AND2X1_RVT U358 ( .A1(n795), .A2(n342), .Y(n705) );
OR3X1_RVT U359 ( .A1(n684), .A2(n686), .A3(n685), .Y(n687) );
OR2X2_RVT U360 ( .A1(n634), .A2(n429), .Y(n343) );
OR2X1_RVT U361 ( .A1(n356), .A2(n346), .Y(n355) );
OR2X2_RVT U362 ( .A1(n427), .A2(n425), .Y(n604) );
OR2X2_RVT U363 ( .A1(opB_reg[1]), .A2(n1023), .Y(n459) );
AND2X1_RVT U364 ( .A1(n703), .A2(n344), .Y(n381) );
INVX1_RVT U365 ( .A(n432), .Y(n461) );
INVX0_RVT U366 ( .A(n517), .Y(n385) );
INVX1_RVT U367 ( .A(n449), .Y(n445) );
INVX0_RVT U368 ( .A(n578), .Y(n436) );
INVX0_RVT U369 ( .A(n533), .Y(n444) );
INVX1_RVT U370 ( .A(n992), .Y(n907) );
OR3X1_RVT U371 ( .A1(n672), .A2(n671), .A3(n670), .Y(n673) );
INVX0_RVT U372 ( .A(n666), .Y(n667) );
OR2X1_RVT U373 ( .A1(n631), .A2(n630), .Y(n394) );
OR3X1_RVT U374 ( .A1(n593), .A2(n592), .A3(n591), .Y(n594) );
```

```
INVX0_RVT U375 ( .A(n915), .Y(n916) );
INVX0_RVT U376 ( .A(n981), .Y(n991) );
AND3X1_RVT U377 ( .A1(n880), .A2(n408), .A3(n626), .Y(n627) );
OR2X1_RVT U378 ( .A1(n548), .A2(n542), .Y(n416) );
INVX0_RVT U379 ( .A(n647), .Y(n651) );
INVX0_RVT U380 ( .A(n523), .Y(n527) );
INVX0_RVT U381 ( .A(n649), .Y(n650) );
OR3X1_RVT U382 ( .A1(n860), .A2(n859), .A3(n858), .Y(n861) );
INVX0_RVT U383 ( .A(n418), .Y(n417) );
OR2X1_RVT U384 ( .A1(n619), .A2(n618), .Y(n395) );
INVX0_RVT U385 ( .A(n414), .Y(n525) );
INVX0_RVT U386 ( .A(n966), .Y(n977) );
INVX0_RVT U387 ( .A(n480), .Y(n479) );
INVX0_RVT U388 ( .A(n854), .Y(n855) );
INVX0_RVT U389 ( .A(n644), .Y(n645) );
INVX0_RVT U390 ( .A(n549), .Y(n422) );
INVX0_RVT U391 ( .A(n564), .Y(n567) );
OR2X1_RVT U392 ( .A1(n432), .A2(n533), .Y(n602) );
INVX0_RVT U393 ( .A(n509), .Y(n648) );
INVX0_RVT U394 ( .A(n887), .Y(n888) );
INVX0_RVT U395 ( .A(n502), .Y(n522) );
INVX0_RVT U396 ( .A(n704), .Y(n344) );
INVX0_RVT U397 ( .A(n839), .Y(n840) );
INVX0_RVT U398 ( .A(n634), .Y(n345) );
OR2X1_RVT U399 ( .A1(n946), .A2(n553), .Y(n635) );
INVX0_RVT U400 ( .A(n661), .Y(n664) );
INVX0_RVT U401 ( .A(n906), .Y(n909) );
```

INVX0_RVT U402 (.A(n836), .Y(n838));
AND2X1_RVT U403 (.A1(n351), .A2(n621), .Y(n473));
INVX0_RVT U404 (.A(n521), .Y(n505));
INVX0_RVT U405 (.A(n696), .Y(n498));
AND2X1_RVT U406 (.A1(n511), .A2(n624), .Y(n510));
INVX0_RVT U407 (.A(n443), .Y(n541));
INVX0_RVT U408 (.A(n441), .Y(n905));
INVX0_RVT U409 (.A(n519), .Y(n515));
INVX0_RVT U410 (.A(n695), .Y(n353));
INVX0_RVT U411 (.A(n520), .Y(n511));
INVX0_RVT U412 (.A(n437), .Y(n660));
INVX0_RVT U413 (.A(n435), .Y(n579));
INVX0_RVT U414 (.A(n420), .Y(n419));
OR2X1_RVT U415 (.A1(n445), .A2(n444), .Y(n443));
OR2X1_RVT U416 (.A1(n445), .A2(n436), .Y(n435));
OR2X1_RVT U417 (.A1(n445), .A2(n442), .Y(n441));
OR2X1_RVT U418 (.A1(n938), .A2(n815), .Y(n872));
OR2X1_RVT U419 (.A1(n1004), .A2(n518), .Y(n349));
INVX0_RVT U420 (.A(n870), .Y(n886));
OR2X1_RVT U421 (.A1(n445), .A2(n438), .Y(n437));
INVX0_RVT U422 (.A(n426), .Y(n425));
INVX0_RVT U423 (.A(n503), .Y(n624));
INVX0_RVT U424 (.A(n970), .Y(n971));
OR2X1_RVT U425 (.A1(n358), .A2(opA_reg[1]), .Y(n412));
INVX0_RVT U426 (.A(n384), .Y(n346));
INVX0_RVT U427 (.A(n830), .Y(n831));
INVX0_RVT U428 (.A(n920), .Y(n843));

INVX0_RVT U429 (.A(n545), .Y(n421));
OR2X1_RVT U430 (.A1(n431), .A2(n444), .Y(n426));
INVX0_RVT U431 (.A(n827), .Y(n835));
OR2X1_RVT U432 (.A1(n825), .A2(n691), .Y(n692));
INVX0_RVT U433 (.A(n625), .Y(n880));
INVX0_RVT U434 (.A(n899), .Y(n825));
AND2X1_RVT U435 (.A1(n463), .A2(n387), .Y(n465));
INVX0_RVT U436 (.A(n451), .Y(n450));
INVX0_RVT U437 (.A(n774), .Y(n828));
NBUFFX2_RVT U438 (.A(n495), .Y(n358));
INVX0_RVT U439 (.A(n554), .Y(n771));
NBUFFX2_RVT U440 (.A(n824), .Y(n993));
INVX0_RVT U441 (.A(n554), .Y(n874));
INVX0_RVT U442 (.A(n653), .Y(n922));
INVX0_RVT U443 (.A(n529), .Y(n978));
INVX0_RVT U444 (.A(n764), .Y(n625));
INVX0_RVT U445 (.A(n370), .Y(n747));
INVX0_RVT U446 (.A(n944), .Y(n457));
INVX0_RVT U447 (.A(n388), .Y(n387));
INVX0_RVT U448 (.A(n818), .Y(n554));
INVX0_RVT U449 (.A(n895), .Y(n973));
INVX0_RVT U450 (.A(n359), .Y(n578));
OR2X1_RVT U451 (.A1(n452), .A2(n1011), .Y(n451));
OR2X1_RVT U452 (.A1(n484), .A2(n535), .Y(n982));
INVX1_RVT U453 (.A(n930), .Y(n533));
NBUFFX2_RVT U454 (.A(n967), .Y(n866));
AND3X1_RVT U455 (.A1(n963), .A2(n766), .A3(n931), .Y(n899));

OR2X1_RVT U456 (.A1(n491), .A2(n490), .Y(n824));
INVX0_RVT U457 (.A(n396), .Y(n362));
NBUFFX2_RVT U458 (.A(n800), .Y(n917));
OR2X1_RVT U459 (.A1(n406), .A2(n371), .Y(n370));
INVX0_RVT U460 (.A(n952), .Y(n455));
OR2X1_RVT U461 (.A1(n959), .A2(n933), .Y(n927));
INVX0_RVT U462 (.A(n926), .Y(n984));
OR2X1_RVT U463 (.A1(opA_reg[2]), .A2(n389), .Y(n388));
NBUFFX2_RVT U464 (.A(n543), .Y(n992));
NBUFFX2_RVT U465 (.A(n683), .Y(n396));
NBUFFX2_RVT U466 (.A(n487), .Y(n707));
NBUFFX2_RVT U467 (.A(n534), .Y(n963));
INVX0_RVT U468 (.A(n950), .Y(n433));
INVX0_RVT U469 (.A(n954), .Y(n603));
INVX0_RVT U470 (.A(n514), .Y(n456));
OR2X1_RVT U471 (.A1(n801), .A2(n1015), .Y(n926));
INVX0_RVT U472 (.A(n801), .Y(n972));
INVX0_RVT U473 (.A(n820), .Y(n424));
INVX0_RVT U474 (.A(n494), .Y(n530));
INVX0_RVT U475 (.A(n662), .Y(n643));
INVX0_RVT U476 (.A(n459), .Y(n934));
INVX0_RVT U477 (.A(n898), .Y(n974));
INVX0_RVT U478 (.A(n514), .Y(n867));
INVX0_RVT U479 (.A(n583), .Y(n621));
INVX0_RVT U480 (.A(n464), .Y(n389));
INVX0_RVT U481 (.A(n719), .Y(n742));
INVX0_RVT U482 (.A(n1004), .Y(n573));

```
INVX0_RVT U483 ( .A(n1016), .Y(n488) );
INVX0_RVT U484 ( .A(opA_reg[1]), .Y(n386) );
INVX0_RVT U485 ( .A(n1018), .Y(n734) );
INVX1_RVT U486 ( .A(n1006), .Y(n489) );
INVX1_RVT U487 ( .A(n1019), .Y(n544) );
INVX0_RVT U488 ( .A(n1018), .Y(n662) );
INVX1_RVT U489 ( .A(n1006), .Y(n532) );
INVX0_RVT U490 ( .A(opA_reg[2]), .Y(n620) );
INVX0_RVT U491 ( .A(opA_reg[0]), .Y(n996) );
INVX0_RVT U492 ( .A(n1005), .Y(n778) );
INVX0_RVT U493 ( .A(n1019), .Y(n654) );
INVX1_RVT U494 ( .A(n1017), .Y(n583) );
INVX0_RVT U495 ( .A(n1005), .Y(n801) );
INVX1_RVT U496 ( .A(n1020), .Y(n727) );
INVX0_RVT U497 ( .A(opA_reg[3]), .Y(n514) );
INVX1_RVT U498 ( .A(n1028), .Y(n683) );
INVX0_RVT U499 ( .A(n1016), .Y(n494) );
AND2X1_RVT U500 ( .A1(n368), .A2(n889), .Y(n887) );
OR2X1_RVT U501 ( .A1(n518), .A2(n844), .Y(n889) );
INVX0_RVT U502 ( .A(n367), .Y(n752) );
INVX0_RVT U503 ( .A(n348), .Y(n847) );
AND2X1_RVT U504 ( .A1(n845), .A2(n847), .Y(n844) );
AND2X1_RVT U505 ( .A1(n471), .A2(n470), .Y(n351) );
AND2X1_RVT U506 ( .A1(n692), .A2(n353), .Y(n352) );
NBUFFX2_RVT U507 ( .A(n802), .Y(n354) );
NBUFFX2_RVT U508 ( .A(opA_reg[1]), .Y(n898) );
AND3X1_RVT U509 ( .A1(n992), .A2(n683), .A3(n1009), .Y(n684) );
```

```
OR2X1_RVT U510 ( .A1(n383), .A2(n346), .Y(n845) );
INVX0_RVT U511 ( .A(n383), .Y(n356) );
NBUFFX2_RVT U512 ( .A(opA_reg[3]), .Y(n357) );
AND2X1_RVT U513 ( .A1(n360), .A2(n948), .Y(n359) );
INVX0_RVT U514 ( .A(n946), .Y(n360) );
OR2X1_RVT U515 ( .A1(n583), .A2(n1012), .Y(n948) );
AND2X1_RVT U516 ( .A1(n842), .A2(n439), .Y(n361) );
OR2X1_RVT U517 ( .A1(n445), .A2(n440), .Y(n439) );
OR2X1_RVT U518 ( .A1(n510), .A2(n512), .Y(n363) );
AND3X1_RVT U519 ( .A1(n622), .A2(n623), .A3(n624), .Y(n628) );
NBUFFX2_RVT U520 ( .A(n742), .Y(n364) );
OR2X1_RVT U521 ( .A1(n417), .A2(n547), .Y(n365) );
OR2X1_RVT U522 ( .A1(n365), .A2(n415), .Y(n550) );
NBUFFX2_RVT U523 ( .A(n752), .Y(n366) );
NBUFFX2_RVT U524 ( .A(n633), .Y(n367) );
OR2X1_RVT U525 ( .A1(n516), .A2(n513), .Y(n368) );
AND2X1_RVT U526 ( .A1(n467), .A2(n466), .Y(n369) );
INVX0_RVT U527 ( .A(B[1]), .Y(n371) );
AND2X1_RVT U528 ( .A1(n474), .A2(n397), .Y(n372) );
OR2X1_RVT U529 ( .A1(n635), .A2(n922), .Y(n636) );
OR2X1_RVT U530 ( .A1(n578), .A2(n559), .Y(n561) );
OR2X1_RVT U531 ( .A1(n946), .A2(n945), .Y(n947) );
AND2X1_RVT U532 ( .A1(n352), .A2(n374), .Y(n373) );
INVX0_RVT U533 ( .A(n690), .Y(n374) );
NBUFFX2_RVT U534 ( .A(n597), .Y(n375) );
INVX0_RVT U535 ( .A(n797), .Y(n719) );
INVX0_RVT U536 ( .A(n1004), .Y(n820) );
```

```
INVX1_RVT U537 ( .A(en), .Y(n797) );
INVX0_RVT U538 ( .A(n375), .Y(n726) );
OR2X1_RVT U539 ( .A1(n708), .A2(n709), .Y(n335) );
OR2X1_RVT U540 ( .A1(n808), .A2(n807), .Y(n332) );
OR2X1_RVT U541 ( .A1(n782), .A2(n527), .Y(n376) );
INVX0_RVT U542 ( .A(n376), .Y(n378) );
OR3X1_RVT U543 ( .A1(n378), .A2(n377), .A3(n416), .Y(n415) );
AND2X1_RVT U544 ( .A1(n526), .A2(n529), .Y(n377) );
AND2X1_RVT U545 ( .A1(n366), .A2(result[1]), .Y(n798) );
AND2X1_RVT U546 ( .A1(n399), .A2(result[2]), .Y(n863) );
AND2X1_RVT U547 ( .A1(n366), .A2(result[3]), .Y(n918) );
AND2X1_RVT U548 ( .A1(n399), .A2(result[0]), .Y(n1001) );
AND2X1_RVT U549 ( .A1(n373), .A2(n379), .Y(n405) );
OR3X1_RVT U550 ( .A1(n782), .A2(n381), .A3(n380), .Y(n379) );
AND2X1_RVT U551 ( .A1(n454), .A2(n704), .Y(n380) );
INVX0_RVT U552 ( .A(n972), .Y(n382) );
AND2X1_RVT U553 ( .A1(n382), .A2(n412), .Y(n383) );
OR2X1_RVT U554 ( .A1(n503), .A2(n386), .Y(n384) );
AND2X1_RVT U555 ( .A1(n464), .A2(n463), .Y(n809) );
OR3X1_RVT U556 ( .A1(n391), .A2(n390), .A3(n394), .Y(n393) );
AND2X1_RVT U557 ( .A1(n617), .A2(n396), .Y(n390) );
AND2X1_RVT U558 ( .A1(n395), .A2(n396), .Y(n391) );
AND2X1_RVT U559 ( .A1(n745), .A2(n393), .Y(n392) );
AND2X1_RVT U560 ( .A1(n474), .A2(n397), .Y(n476) );
OR2X1_RVT U561 ( .A1(n621), .A2(n472), .Y(n397) );
OR2X1_RVT U562 ( .A1(n521), .A2(n644), .Y(n524) );
NBUFFX2_RVT U563 ( .A(n821), .Y(n398) );
```

AND2X1_RVT U564 (.A1(n522), .A2(n524), .Y(n414));

INVX0_RVT U565 (.A(n797), .Y(n597));

INVX0_RVT U566 (.A(n385), .Y(n413));

OR2X1_RVT U567 (.A1(n753), .A2(n754), .Y(n321));

OR2X1_RVT U568 (.A1(n713), .A2(n714), .Y(n327));

OR2X1_RVT U569 (.A1(n740), .A2(n741), .Y(n324));

NBUFFX2_RVT U570 (.A(n354), .Y(n399));

NBUFFX2_RVT U571 (.A(n715), .Y(n400));

INVX0_RVT U572 (.A(n405), .Y(n404));

OR2X1_RVT U573 (.A1(n688), .A2(n687), .Y(n689));

NBUFFX2_RVT U574 (.A(n759), .Y(n401));

NBUFFX2_RVT U575 (.A(n364), .Y(n402));

NBUFFX2_RVT U576 (.A(n496), .Y(n403));

NBUFFX2_RVT U577 (.A(n742), .Y(n406));

NBUFFX2_RVT U578 (.A(n1015), .Y(n407));

OR2X1_RVT U579 (.A1(n414), .A2(n696), .Y(n703));

NBUFFX2_RVT U580 (.A(opB_reg[7]), .Y(n408));

OR2X1_RVT U581 (.A1(n564), .A2(n520), .Y(n646));

NBUFFX2_RVT U582 (.A(n726), .Y(n410));

AND2X1_RVT U583 (.A1(n384), .A2(n412), .Y(n411));

AND2X1_RVT U584 (.A1(n422), .A2(n419), .Y(n418));

OR2X1_RVT U585 (.A1(n546), .A2(n421), .Y(n420));

INVX0_RVT U586 (.A(n423), .Y(n938));

AND2X1_RVT U587 (.A1(n811), .A2(n816), .Y(n815));

AND2X1_RVT U588 (.A1(n940), .A2(n423), .Y(n811));

OR2X1_RVT U589 (.A1(opB_reg[2]), .A2(n424), .Y(n423));

OR2X1_RVT U590 (.A1(n431), .A2(n430), .Y(n429));

INVX0_RVT U591 (.A(n433), .Y(n430));

INVX0_RVT U592 (.A(n603), .Y(n431));

INVX0_RVT U593 (.A(n653), .Y(n438));

INVX0_RVT U594 (.A(n843), .Y(n440));

INVX0_RVT U595 (.A(n876), .Y(n442));

OR2X1_RVT U596 (.A1(n449), .A2(n985), .Y(n986));

AND2X1_RVT U597 (.A1(n449), .A2(n434), .Y(n693));

NBUFFX2_RVT U598 (.A(n927), .Y(n434));

INVX0_RVT U599 (.A(n446), .Y(n769));

AND2X1_RVT U600 (.A1(n453), .A2(n447), .Y(n446));

INVX0_RVT U601 (.A(n448), .Y(n447));

OR2X1_RVT U602 (.A1(n449), .A2(n767), .Y(n448));

AND2X1_RVT U603 (.A1(n532), .A2(n450), .Y(n449));

INVX0_RVT U604 (.A(n531), .Y(n452));

INVX0_RVT U605 (.A(n768), .Y(n453));

AND2X1_RVT U606 (.A1(n648), .A2(n646), .Y(n644));

INVX0_RVT U607 (.A(n703), .Y(n454));

NBUFFX2_RVT U608 (.A(n811), .Y(n920));

OR2X1_RVT U609 (.A1(n580), .A2(n579), .Y(n581));

NBUFFX2_RVT U610 (.A(n726), .Y(n802));

NBUFFX2_RVT U611 (.A(n531), .Y(n765));

AND4X1_RVT U612 (.A1(n532), .A2(n487), .A3(n531), .A4(n530), .Y(n491));

NBUFFX2_RVT U613 (.A(n532), .Y(n964));

NBUFFX2_RVT U614 (.A(n1024), .Y(n531));

NBUFFX2_RVT U615 (.A(n802), .Y(n739));

NBUFFX2_RVT U616 (.A(n364), .Y(n722));

NBUFFX2_RVT U617 (.A(n1025), .Y(n715));

OR2X1_RVT U618 (.A1(n832), .A2(n831), .Y(n833));

INVX0_RVT U619 (.A(n833), .Y(n834));

INVX0_RVT U620 (.A(n679), .Y(n680));

NBUFFX2_RVT U621 (.A(n982), .Y(n821));

NBUFFX2_RVT U622 (.A(n620), .Y(n829));

NBUFFX2_RVT U623 (.A(n752), .Y(n806));

INVX0_RVT U624 (.A(rst), .Y(n1003));

AND2X1_RVT U625 (.A1(result[6]), .A2(n722), .Y(n552));

NBUFFX2_RVT U626 (.A(n375), .Y(n710));

NBUFFX2_RVT U627 (.A(n710), .Y(n999));

AND2X1_RVT U628 (.A1(n1027), .A2(n1014), .Y(n954));

OR2X1_RVT U629 (.A1(n544), .A2(n1014), .Y(n956));

AND2X1_RVT U630 (.A1(n603), .A2(n956), .Y(n930));

AND2X1_RVT U631 (.A1(n1026), .A2(n1013), .Y(n950));

OR2X1_RVT U632 (.A1(n662), .A2(n1013), .Y(n952));

OR2X1_RVT U633 (.A1(n950), .A2(n455), .Y(n653));

AND2X1_RVT U634 (.A1(n1025), .A2(n1012), .Y(n946));

AND2X1_RVT U635 (.A1(n357), .A2(n1010), .Y(n942));

OR2X1_RVT U636 (.A1(n456), .A2(n1010), .Y(n944));

OR2X1_RVT U637 (.A1(n942), .A2(n457), .Y(n876));

INVX0_RVT U638 (.A(n876), .Y(n921));

AND2X1_RVT U639 (.A1(n1023), .A2(opB_reg[1]), .Y(n458));

INVX0_RVT U640 (.A(n458), .Y(n935));

AND2X1_RVT U641 (.A1(n459), .A2(n935), .Y(n759));

AND2X1_RVT U642 (.A1(n401), .A2(n926), .Y(n460));

OR2X1_RVT U643 (.A1(n934), .A2(n460), .Y(n816));

OR2X1_RVT U644 (.A1(n820), .A2(n1007), .Y(n940));

AND2X1_RVT U645 (.A1(n921), .A2(n872), .Y(n871));

OR2X1_RVT U646 (.A1(n942), .A2(n871), .Y(n555));

AND2X1_RVT U647 (.A1(n359), .A2(n555), .Y(n553));

AND2X1_RVT U648 (.A1(n922), .A2(n635), .Y(n634));

AND2X1_RVT U649 (.A1(n1006), .A2(n1022), .Y(n766));

AND3X1_RVT U650 (.A1(n1011), .A2(n766), .A3(n765), .Y(n818));

OR2X1_RVT U651 (.A1(n461), .A2(n930), .Y(n462));

AND3X1_RVT U652 (.A1(n602), .A2(n874), .A3(n462), .Y(n549));

OR2X1_RVT U653 (.A1(n1023), .A2(n1021), .Y(n464));

OR2X1_RVT U654 (.A1(n465), .A2(n1007), .Y(n467));

OR2X1_RVT U655 (.A1(n809), .A2(n620), .Y(n466));

AND2X1_RVT U656 (.A1(n467), .A2(n466), .Y(n469));

AND2X1_RVT U657 (.A1(n369), .A2(n1008), .Y(n468));

OR2X1_RVT U658 (.A1(n468), .A2(n1010), .Y(n471));

OR2X1_RVT U659 (.A1(n1008), .A2(n469), .Y(n470));

AND2X1_RVT U660 (.A1(n471), .A2(n470), .Y(n472));

OR2X1_RVT U661 (.A1(n473), .A2(n1012), .Y(n474));

AND2X1_RVT U662 (.A1(n372), .A2(n643), .Y(n475));

OR2X1_RVT U663 (.A1(n475), .A2(n1013), .Y(n478));

OR2X1_RVT U664 (.A1(n476), .A2(n643), .Y(n477));

AND2X1_RVT U665 (.A1(n478), .A2(n477), .Y(n608));

NBUFFX2_RVT U666 (.A(n608), .Y(n480));

OR2X1_RVT U667 (.A1(n479), .A2(n533), .Y(n482));

NBUFFX2_RVT U668 (.A(n530), .Y(n931));

AND3X1_RVT U669 (.A1(n766), .A2(n765), .A3(n931), .Y(n764));

OR2X1_RVT U670 (.A1(n480), .A2(n930), .Y(n481));

AND3X1_RVT U671 (.A1(n482), .A2(n764), .A3(n481), .Y(n548));

NBUFFX2_RVT U672 (.A(opcode_reg[1]), .Y(n487));
INVX0_RVT U673 (.A(n489), .Y(n483));
AND3X1_RVT U674 (.A1(n707), .A2(n483), .A3(n765), .Y(n967));
AND2X1_RVT U675 (.A1(n654), .A2(n866), .Y(n485));
NBUFFX2_RVT U676 (.A(opcode_reg[3]), .Y(n534));
AND4X1_RVT U677 (.A1(n534), .A2(n964), .A3(n707), .A4(n1011), .Y(n484));
NBUFFX2_RVT U678 (.A(n488), .Y(n960));
AND4X1_RVT U679 (.A1(n707), .A2(n960), .A3(n1024), .A4(n483), .Y(n535));
OR2X1_RVT U680 (.A1(n485), .A2(n398), .Y(n486));
AND2X1_RVT U681 (.A1(n486), .A2(opB_reg[6]), .Y(n547));
AND3X1_RVT U682 (.A1(n489), .A2(n488), .A3(n531), .Y(n543));
AND2X1_RVT U683 (.A1(n543), .A2(n1022), .Y(n490));
AND2X1_RVT U684 (.A1(n691), .A2(n824), .Y(n546));
INVX0_RVT U685 (.A(opcode_reg[2]), .Y(n492));
AND2X1_RVT U686 (.A1(n492), .A2(opcode_reg[3]), .Y(n493));
AND2X1_RVT U687 (.A1(opcode_reg[1]), .A2(n493), .Y(n528));
AND2X1_RVT U688 (.A1(n494), .A2(n528), .Y(n495));
INVX0_RVT U689 (.A(n495), .Y(n503));
INVX0_RVT U690 (.A(n503), .Y(n517));
NBUFFX2_RVT U691 (.A(n517), .Y(n496));
AND2X1_RVT U692 (.A1(n403), .A2(n654), .Y(n696));
AND2X1_RVT U693 (.A1(n498), .A2(n624), .Y(n497));
INVX0_RVT U694 (.A(n497), .Y(n501));
AND2X1_RVT U695 (.A1(n654), .A2(n498), .Y(n499));
INVX0_RVT U696 (.A(n499), .Y(n500));
AND2X1_RVT U697 (.A1(n501), .A2(n500), .Y(n502));
AND2X1_RVT U698 (.A1(n662), .A2(n413), .Y(n521));

AND2X1_RVT U699 (.A1(n505), .A2(n413), .Y(n504));
INVX0_RVT U700 (.A(n504), .Y(n508));
AND2X1_RVT U701 (.A1(n1026), .A2(n505), .Y(n506));
INVX0_RVT U702 (.A(n506), .Y(n507));
AND2X1_RVT U703 (.A1(n508), .A2(n507), .Y(n509));
AND2X1_RVT U704 (.A1(n624), .A2(n583), .Y(n520));
AND2X1_RVT U705 (.A1(n715), .A2(n511), .Y(n512));
AND2X1_RVT U706 (.A1(n357), .A2(n496), .Y(n519));
AND2X1_RVT U707 (.A1(n515), .A2(n403), .Y(n513));
AND2X1_RVT U708 (.A1(n867), .A2(n515), .Y(n516));
AND2X1_RVT U709 (.A1(n573), .A2(n358), .Y(n518));
OR2X1_RVT U710 (.A1(n887), .A2(n519), .Y(n565));
AND2X1_RVT U711 (.A1(n363), .A2(n565), .Y(n564));
AND2X1_RVT U712 (.A1(n522), .A2(n525), .Y(n523));
AND2X1_RVT U713 (.A1(n525), .A2(n524), .Y(n526));
NBUFFX2_RVT U714 (.A(n528), .Y(n529));
AND3X1_RVT U715 (.A1(n963), .A2(n960), .A3(n766), .Y(n895));
NBUFFX2_RVT U716 (.A(n895), .Y(n826));
AND2X1_RVT U717 (.A1(n727), .A2(n826), .Y(n539));
NBUFFX2_RVT U718 (.A(n535), .Y(n774));
NBUFFX2_RVT U719 (.A(n774), .Y(n969));
AND2X1_RVT U720 (.A1(n1027), .A2(n969), .Y(n538));
AND4X1_RVT U721 (.A1(n964), .A2(n963), .A3(n707), .A4(n931), .Y(n897));
AND2X1_RVT U722 (.A1(n1025), .A2(n897), .Y(n537));
AND2X1_RVT U723 (.A1(n734), .A2(n899), .Y(n536));
OR4X1_RVT U724 (.A1(n539), .A2(n538), .A3(n537), .A4(n536), .Y(n540));
OR2X1_RVT U725 (.A1(n541), .A2(n540), .Y(n542));

OR3X1_RVT U726 (.A1(n544), .A2(n907), .A3(opB_reg[6]), .Y(n545));

AND2X1_RVT U727 (.A1(n999), .A2(n550), .Y(n551));

OR2X1_RVT U728 (.A1(n552), .A2(n551), .Y(n315));

AND2X1_RVT U729 (.A1(result[4]), .A2(n722), .Y(n596));

INVX0_RVT U730 (.A(n553), .Y(n557));

OR2X1_RVT U731 (.A1(n555), .A2(n436), .Y(n556));

AND3X1_RVT U732 (.A1(n557), .A2(n771), .A3(n556), .Y(n593));

NBUFFX2_RVT U733 (.A(n472), .Y(n558));

INVX0_RVT U734 (.A(n558), .Y(n559));

OR2X1_RVT U735 (.A1(n558), .A2(n436), .Y(n560));

AND3X1_RVT U736 (.A1(n561), .A2(n760), .A3(n560), .Y(n592));

AND2X1_RVT U737 (.A1(n1025), .A2(n866), .Y(n562));

OR2X1_RVT U738 (.A1(n562), .A2(n821), .Y(n563));

AND2X1_RVT U739 (.A1(n563), .A2(opB_reg[4]), .Y(n590));

AND2X1_RVT U740 (.A1(n621), .A2(n824), .Y(n589));

AND2X1_RVT U741 (.A1(n567), .A2(n565), .Y(n566));

INVX0_RVT U742 (.A(n566), .Y(n570));

AND2X1_RVT U743 (.A1(n567), .A2(n363), .Y(n568));

INVX0_RVT U744 (.A(n568), .Y(n569));

AND2X1_RVT U745 (.A1(n570), .A2(n569), .Y(n571));

OR2X1_RVT U746 (.A1(n571), .A2(n782), .Y(n572));

INVX0_RVT U747 (.A(n572), .Y(n587));

AND2X1_RVT U748 (.A1(n573), .A2(n897), .Y(n582));

NBUFFX2_RVT U749 (.A(n867), .Y(n896));

AND2X1_RVT U750 (.A1(n896), .A2(n899), .Y(n577));

AND2X1_RVT U751 (.A1(n1026), .A2(n895), .Y(n575));

AND2X1_RVT U752 (.A1(n400), .A2(n969), .Y(n574));

```
OR2X1_RVT U753 ( .A1(n575), .A2(n574), .Y(n576) );
OR2X1_RVT U754 ( .A1(n577), .A2(n576), .Y(n580) );
OR2X1_RVT U755 ( .A1(n582), .A2(n581), .Y(n586) );
OR3X1_RVT U756 ( .A1(n583), .A2(n907), .A3(opB_reg[4]), .Y(n584) );
INVX0_RVT U757 ( .A(n584), .Y(n585) );
OR3X1_RVT U758 ( .A1(n587), .A2(n586), .A3(n585), .Y(n588) );
OR3X1_RVT U759 ( .A1(n590), .A2(n589), .A3(n588), .Y(n591) );
NBUFFX2_RVT U760 ( .A(n719), .Y(n718) );
NBUFFX2_RVT U761 ( .A(n718), .Y(n805) );
AND2X1_RVT U762 ( .A1(n594), .A2(n805), .Y(n595) );
OR2X1_RVT U763 ( .A1(n596), .A2(n595), .Y(n313) );
AND2X1_RVT U764 ( .A1(overflow), .A2(n722), .Y(n632) );
NBUFFX2_RVT U765 ( .A(n597), .Y(n633) );
NBUFFX2_RVT U766 ( .A(n710), .Y(n745) );
OR2X1_RVT U767 ( .A1(n1011), .A2(n782), .Y(n598) );
INVX0_RVT U768 ( .A(n598), .Y(n601) );
AND4X1_RVT U769 ( .A1(n400), .A2(opA_reg[2]), .A3(n896), .A4(n734), .Y(n599)
    );
AND4X1_RVT U770 ( .A1(n778), .A2(n898), .A3(n1027), .A4(n599), .Y(n600) );
AND2X1_RVT U771 ( .A1(n601), .A2(n600), .Y(n619) );
AND2X1_RVT U772 ( .A1(n683), .A2(opB_reg[7]), .Y(n959) );
AND2X1_RVT U773 ( .A1(n1009), .A2(n727), .Y(n933) );
AND2X1_RVT U774 ( .A1(n604), .A2(n927), .Y(n605) );
INVX0_RVT U775 ( .A(n605), .Y(n606) );
AND3X1_RVT U776 ( .A1(n607), .A2(n771), .A3(n606), .Y(n686) );
AND2X1_RVT U777 ( .A1(n408), .A2(n686), .Y(n618) );
AND2X1_RVT U778 ( .A1(n608), .A2(n691), .Y(n609) );
```

```
OR2X1_RVT U779 ( .A1(n609), .A2(n1014), .Y(n611) );
OR2X1_RVT U780 ( .A1(n608), .A2(n691), .Y(n610) );
AND2X1_RVT U781 ( .A1(n610), .A2(n611), .Y(n612) );
AND2X1_RVT U782 ( .A1(n927), .A2(n612), .Y(n613) );
INVX0_RVT U783 ( .A(n613), .Y(n614) );
AND2X1_RVT U784 ( .A1(n615), .A2(n614), .Y(n626) );
INVX0_RVT U785 ( .A(n626), .Y(n616) );
AND2X1_RVT U786 ( .A1(n760), .A2(n616), .Y(n685) );
AND2X1_RVT U787 ( .A1(n685), .A2(n1009), .Y(n617) );
AND4X1_RVT U788 ( .A1(n621), .A2(n829), .A3(n1008), .A4(n643), .Y(n623) );
AND3X1_RVT U789 ( .A1(n996), .A2(n974), .A3(n691), .Y(n622) );
OR2X1_RVT U790 ( .A1(n628), .A2(n627), .Y(n629) );
AND2X1_RVT U791 ( .A1(n362), .A2(n629), .Y(n630) );
OR2X1_RVT U792 ( .A1(n632), .A2(n392), .Y(n317) );
AND2X1_RVT U793 ( .A1(result[5]), .A2(n402), .Y(n675) );
NBUFFX2_RVT U794 ( .A(n633), .Y(n800) );
NBUFFX2_RVT U795 ( .A(n476), .Y(n637) );
INVX0_RVT U796 ( .A(n637), .Y(n638) );
OR2X1_RVT U797 ( .A1(n638), .A2(n653), .Y(n640) );
OR2X1_RVT U798 ( .A1(n637), .A2(n922), .Y(n639) );
AND3X1_RVT U799 ( .A1(n640), .A2(n880), .A3(n639), .Y(n671) );
AND2X1_RVT U800 ( .A1(n1026), .A2(n866), .Y(n641) );
OR2X1_RVT U801 ( .A1(n641), .A2(n398), .Y(n642) );
AND2X1_RVT U802 ( .A1(n642), .A2(opB_reg[5]), .Y(n669) );
AND2X1_RVT U803 ( .A1(n643), .A2(n993), .Y(n668) );
AND2X1_RVT U804 ( .A1(n645), .A2(n646), .Y(n647) );
AND2X1_RVT U805 ( .A1(n648), .A2(n645), .Y(n649) );
```

AND2X1_RVT U806 (.A1(n651), .A2(n650), .Y(n652));

OR2X1_RVT U807 (.A1(n652), .A2(n978), .Y(n665));

AND2X1_RVT U808 (.A1(n654), .A2(n826), .Y(n658));

AND2X1_RVT U809 (.A1(n734), .A2(n774), .Y(n657));

AND2X1_RVT U810 (.A1(n867), .A2(n897), .Y(n656));

AND2X1_RVT U811 (.A1(n400), .A2(n899), .Y(n655));

OR4X1_RVT U812 (.A1(n658), .A2(n657), .A3(n656), .A4(n655), .Y(n659));

OR2X1_RVT U813 (.A1(n660), .A2(n659), .Y(n661));

OR3X1_RVT U814 (.A1(n662), .A2(n907), .A3(opB_reg[5]), .Y(n663));

AND3X1_RVT U815 (.A1(n665), .A2(n664), .A3(n663), .Y(n666));

OR3X1_RVT U816 (.A1(n669), .A2(n668), .A3(n667), .Y(n670));

AND2X1_RVT U817 (.A1(n917), .A2(n673), .Y(n674));

OR2X1_RVT U818 (.A1(n675), .A2(n674), .Y(n314));

AND2X1_RVT U819 (.A1(result[7]), .A2(n366), .Y(n706));

AND2X1_RVT U820 (.A1(n683), .A2(n824), .Y(n676));

INVX0_RVT U821 (.A(n676), .Y(n681));

AND2X1_RVT U822 (.A1(n408), .A2(n967), .Y(n677));

OR3X1_RVT U823 (.A1(n826), .A2(n774), .A3(n677), .Y(n678));

AND2X1_RVT U824 (.A1(n362), .A2(n678), .Y(n679));

AND2X1_RVT U825 (.A1(n681), .A2(n680), .Y(n682));

INVX0_RVT U826 (.A(n682), .Y(n688));

AND2X1_RVT U827 (.A1(n408), .A2(n821), .Y(n690));

AND2X1_RVT U828 (.A1(n734), .A2(n897), .Y(n694));

OR2X1_RVT U829 (.A1(n694), .A2(n693), .Y(n695));

AND2X1_RVT U830 (.A1(n1028), .A2(n403), .Y(n697));

INVX0_RVT U831 (.A(n697), .Y(n699));

AND2X1_RVT U832 (.A1(n699), .A2(n413), .Y(n698));

```
INVX0_RVT U833 ( .A(n698), .Y(n702) );
AND2X1_RVT U834 ( .A1(n727), .A2(n699), .Y(n700) );
INVX0_RVT U835 ( .A(n700), .Y(n701) );
AND2X1_RVT U836 ( .A1(n702), .A2(n701), .Y(n704) );
NBUFFX2_RVT U837 ( .A(n718), .Y(n795) );
AND2X1_RVT U838 ( .A1(n795), .A2(opcode[1]), .Y(n709) );
AND2X1_RVT U839 ( .A1(n707), .A2(n410), .Y(n708) );
NBUFFX2_RVT U840 ( .A(n367), .Y(n862) );
AND2X1_RVT U841 ( .A1(n862), .A2(opcode[2]), .Y(n712) );
AND2X1_RVT U842 ( .A1(n964), .A2(n410), .Y(n711) );
OR2X1_RVT U843 ( .A1(n711), .A2(n712), .Y(n336) );
AND2X1_RVT U844 ( .A1(n805), .A2(A[1]), .Y(n714) );
AND2X1_RVT U845 ( .A1(n898), .A2(n402), .Y(n713) );
AND2X1_RVT U846 ( .A1(n745), .A2(A[4]), .Y(n717) );
AND2X1_RVT U847 ( .A1(n410), .A2(n400), .Y(n716) );
OR2X1_RVT U848 ( .A1(n717), .A2(n716), .Y(n330) );
NBUFFX2_RVT U849 ( .A(n718), .Y(n725) );
AND2X1_RVT U850 ( .A1(n725), .A2(A[3]), .Y(n721) );
AND2X1_RVT U851 ( .A1(n406), .A2(n896), .Y(n720) );
OR2X1_RVT U852 ( .A1(n721), .A2(n720), .Y(n329) );
AND2X1_RVT U853 ( .A1(n745), .A2(A[2]), .Y(n724) );
AND2X1_RVT U854 ( .A1(n722), .A2(n573), .Y(n723) );
OR2X1_RVT U855 ( .A1(n724), .A2(n723), .Y(n328) );
AND2X1_RVT U856 ( .A1(n725), .A2(A[7]), .Y(n729) );
AND2X1_RVT U857 ( .A1(n739), .A2(n727), .Y(n728) );
OR2X1_RVT U858 ( .A1(n729), .A2(n728), .Y(n333) );
AND2X1_RVT U859 ( .A1(n999), .A2(opcode[0]), .Y(n731) );
```

AND2X1_RVT U860 (.A1(n960), .A2(n410), .Y(n730));
OR2X1_RVT U861 (.A1(n731), .A2(n730), .Y(n334));
AND2X1_RVT U862 (.A1(n917), .A2(opcode[3]), .Y(n733));
AND2X1_RVT U863 (.A1(n963), .A2(n354), .Y(n732));
OR2X1_RVT U864 (.A1(n733), .A2(n732), .Y(n337));
AND2X1_RVT U865 (.A1(n725), .A2(A[5]), .Y(n736));
AND2X1_RVT U866 (.A1(n739), .A2(n734), .Y(n735));
OR2X1_RVT U867 (.A1(n736), .A2(n735), .Y(n331));
AND2X1_RVT U868 (.A1(n862), .A2(B[7]), .Y(n738));
AND2X1_RVT U869 (.A1(n408), .A2(n402), .Y(n737));
OR2X1_RVT U870 (.A1(n738), .A2(n737), .Y(n325));
AND2X1_RVT U871 (.A1(n805), .A2(B[6]), .Y(n741));
AND2X1_RVT U872 (.A1(opB_reg[6]), .A2(n739), .Y(n740));
AND2X1_RVT U873 (.A1(n800), .A2(B[5]), .Y(n744));
AND2X1_RVT U874 (.A1(opB_reg[5]), .A2(n402), .Y(n743));
OR2X1_RVT U875 (.A1(n744), .A2(n743), .Y(n323));
NBUFFX2_RVT U876 (.A(opB_reg[1]), .Y(n772));
AND2X1_RVT U877 (.A1(n772), .A2(n406), .Y(n746));
OR2X1_RVT U878 (.A1(n747), .A2(n746), .Y(n319));
AND2X1_RVT U879 (.A1(n862), .A2(B[4]), .Y(n749));
AND2X1_RVT U880 (.A1(opB_reg[4]), .A2(n806), .Y(n748));
OR2X1_RVT U881 (.A1(n749), .A2(n748), .Y(n322));
AND2X1_RVT U882 (.A1(n725), .A2(B[0]), .Y(n751));
AND2X1_RVT U883 (.A1(opB_reg[0]), .A2(n806), .Y(n750));
OR2X1_RVT U884 (.A1(n751), .A2(n750), .Y(n318));
AND2X1_RVT U885 (.A1(n795), .A2(B[3]), .Y(n754));
AND2X1_RVT U886 (.A1(opB_reg[3]), .A2(n406), .Y(n753));

```
AND2X1_RVT U887 ( .A1(n999), .A2(B[2]), .Y(n756) );
AND2X1_RVT U888 ( .A1(opB_reg[2]), .A2(n806), .Y(n755) );
OR2X1_RVT U889 ( .A1(n756), .A2(n755), .Y(n320) );
AND2X1_RVT U890 ( .A1(n992), .A2(n1021), .Y(n757) );
OR2X1_RVT U891 ( .A1(n757), .A2(n993), .Y(n758) );
AND2X1_RVT U892 ( .A1(n974), .A2(n758), .Y(n794) );
NBUFFX2_RVT U893 ( .A(n401), .Y(n923) );
AND3X1_RVT U894 ( .A1(n760), .A2(opB_reg[0]), .A3(n923), .Y(n762) );
INVX0_RVT U895 ( .A(n923), .Y(n770) );
AND2X1_RVT U896 ( .A1(n874), .A2(n770), .Y(n761) );
OR3X1_RVT U897 ( .A1(n899), .A2(n762), .A3(n761), .Y(n763) );
AND2X1_RVT U898 ( .A1(n778), .A2(n763), .Y(n793) );
AND2X1_RVT U899 ( .A1(n764), .A2(n996), .Y(n768) );
AND2X1_RVT U900 ( .A1(n766), .A2(n765), .Y(n985) );
AND2X1_RVT U901 ( .A1(n985), .A2(n407), .Y(n767) );
AND2X1_RVT U902 ( .A1(n770), .A2(n769), .Y(n791) );
AND3X1_RVT U903 ( .A1(n923), .A2(n771), .A3(n984), .Y(n790) );
AND2X1_RVT U904 ( .A1(n772), .A2(n398), .Y(n789) );
INVX0_RVT U905 ( .A(n967), .Y(n773) );
OR2X1_RVT U906 ( .A1(n1021), .A2(n773), .Y(n775) );
AND2X1_RVT U907 ( .A1(n775), .A2(n828), .Y(n776) );
OR2X1_RVT U908 ( .A1(n974), .A2(n776), .Y(n786) );
AND2X1_RVT U909 ( .A1(n355), .A2(n411), .Y(n777) );
INVX0_RVT U910 ( .A(n777), .Y(n781) );
AND2X1_RVT U911 ( .A1(n382), .A2(n355), .Y(n779) );
INVX0_RVT U912 ( .A(n779), .Y(n780) );
AND2X1_RVT U913 ( .A1(n781), .A2(n780), .Y(n783) );
```

OR2X1_RVT U914 (.A1(n783), .A2(n782), .Y(n785));

OR2X1_RVT U915 (.A1(n829), .A2(n973), .Y(n784));

AND3X1_RVT U916 (.A1(n786), .A2(n785), .A3(n784), .Y(n787));

INVX0_RVT U917 (.A(n787), .Y(n788));

OR4X1_RVT U918 (.A1(n791), .A2(n790), .A3(n789), .A4(n788), .Y(n792));

OR3X1_RVT U919 (.A1(n794), .A2(n793), .A3(n792), .Y(n796));

AND2X1_RVT U920 (.A1(n796), .A2(n795), .Y(n799));

OR2X1_RVT U921 (.A1(n799), .A2(n798), .Y(n310));

AND2X1_RVT U922 (.A1(n800), .A2(A[0]), .Y(n804));

AND2X1_RVT U923 (.A1(n778), .A2(n752), .Y(n803));

OR2X1_RVT U924 (.A1(n804), .A2(n803), .Y(n326));

AND2X1_RVT U925 (.A1(n805), .A2(A[6]), .Y(n808));

AND2X1_RVT U926 (.A1(n354), .A2(n1027), .Y(n807));

NBUFFX2_RVT U927 (.A(n809), .Y(n812));

INVX0_RVT U928 (.A(n812), .Y(n810));

OR2X1_RVT U929 (.A1(n843), .A2(n810), .Y(n814));

OR2X1_RVT U930 (.A1(n812), .A2(n920), .Y(n813));

AND3X1_RVT U931 (.A1(n814), .A2(n880), .A3(n813), .Y(n860));

INVX0_RVT U932 (.A(n815), .Y(n819));

AND3X1_RVT U933 (.A1(n819), .A2(n818), .A3(n817), .Y(n859));

AND2X1_RVT U934 (.A1(n866), .A2(n820), .Y(n822));

OR2X1_RVT U935 (.A1(n822), .A2(n398), .Y(n823));

AND2X1_RVT U936 (.A1(n823), .A2(opB_reg[2]), .Y(n857));

AND2X1_RVT U937 (.A1(n829), .A2(n824), .Y(n856));

OR2X1_RVT U938 (.A1(n825), .A2(n974), .Y(n841));

AND2X1_RVT U939 (.A1(n867), .A2(n826), .Y(n827));

AND3X1_RVT U940 (.A1(n992), .A2(n829), .A3(n1007), .Y(n832));

OR2X1_RVT U941 (.A1(n829), .A2(n828), .Y(n830));

AND2X1_RVT U942 (.A1(n835), .A2(n834), .Y(n836));

AND2X1_RVT U943 (.A1(n778), .A2(n897), .Y(n837));

OR2X1_RVT U944 (.A1(n838), .A2(n837), .Y(n839));

AND2X1_RVT U945 (.A1(n841), .A2(n840), .Y(n842));

INVX0_RVT U946 (.A(n844), .Y(n848));

AND2X1_RVT U947 (.A1(n848), .A2(n845), .Y(n846));

INVX0_RVT U948 (.A(n846), .Y(n851));

AND2X1_RVT U949 (.A1(n848), .A2(n847), .Y(n849));

INVX0_RVT U950 (.A(n849), .Y(n850));

AND2X1_RVT U951 (.A1(n851), .A2(n850), .Y(n852));

OR2X1_RVT U952 (.A1(n852), .A2(n978), .Y(n853));

AND2X1_RVT U953 (.A1(n361), .A2(n853), .Y(n854));

OR3X1_RVT U954 (.A1(n857), .A2(n856), .A3(n855), .Y(n858));

AND2X1_RVT U955 (.A1(n861), .A2(n862), .Y(n864));

OR2X1_RVT U956 (.A1(n863), .A2(n864), .Y(n311));

AND2X1_RVT U957 (.A1(n1008), .A2(n993), .Y(n865));

INVX0_RVT U958 (.A(n865), .Y(n914));

AND2X1_RVT U959 (.A1(n867), .A2(n866), .Y(n868));

OR2X1_RVT U960 (.A1(n868), .A2(n982), .Y(n869));

AND2X1_RVT U961 (.A1(n869), .A2(opB_reg[3]), .Y(n870));

INVX0_RVT U962 (.A(n871), .Y(n875));

OR2X1_RVT U963 (.A1(n872), .A2(n921), .Y(n873));

AND3X1_RVT U964 (.A1(n875), .A2(n874), .A3(n873), .Y(n883));

NBUFFX2_RVT U965 (.A(n469), .Y(n877));

INVX0_RVT U966 (.A(n877), .Y(n878));

OR2X1_RVT U967 (.A1(n876), .A2(n878), .Y(n881));

OR2X1_RVT U968 (.A1(n921), .A2(n877), .Y(n879));

AND3X1_RVT U969 (.A1(n881), .A2(n880), .A3(n879), .Y(n882));

OR2X1_RVT U970 (.A1(n883), .A2(n882), .Y(n884));

INVX0_RVT U971 (.A(n884), .Y(n885));

AND2X1_RVT U972 (.A1(n886), .A2(n885), .Y(n912));

AND2X1_RVT U973 (.A1(n888), .A2(n889), .Y(n890));

INVX0_RVT U974 (.A(n890), .Y(n893));

AND2X1_RVT U975 (.A1(n888), .A2(n368), .Y(n891));

INVX0_RVT U976 (.A(n891), .Y(n892));

AND2X1_RVT U977 (.A1(n893), .A2(n892), .Y(n894));

OR2X1_RVT U978 (.A1(n894), .A2(n978), .Y(n910));

AND2X1_RVT U979 (.A1(n400), .A2(n895), .Y(n903));

AND2X1_RVT U980 (.A1(n896), .A2(n969), .Y(n902));

AND2X1_RVT U981 (.A1(n898), .A2(n897), .Y(n901));

AND2X1_RVT U982 (.A1(opA_reg[2]), .A2(n899), .Y(n900));

OR4X1_RVT U983 (.A1(n903), .A2(n902), .A3(n901), .A4(n900), .Y(n904));

OR2X1_RVT U984 (.A1(n905), .A2(n904), .Y(n906));

OR3X1_RVT U985 (.A1(n456), .A2(n907), .A3(opB_reg[3]), .Y(n908));

AND3X1_RVT U986 (.A1(n910), .A2(n909), .A3(n908), .Y(n911));

AND2X1_RVT U987 (.A1(n912), .A2(n911), .Y(n913));

AND2X1_RVT U988 (.A1(n914), .A2(n913), .Y(n915));

AND2X1_RVT U989 (.A1(n917), .A2(n916), .Y(n919));

OR2X1_RVT U990 (.A1(n919), .A2(n918), .Y(n312));

AND4X1_RVT U991 (.A1(n923), .A2(n922), .A3(n442), .A4(n920), .Y(n925));

AND2X1_RVT U992 (.A1(n778), .A2(n407), .Y(n983));

INVX0_RVT U993 (.A(n983), .Y(n924));

AND4X1_RVT U994 (.A1(n926), .A2(n925), .A3(n924), .A4(n436), .Y(n929));

```
INVX0_RVT U995 ( .A(n927), .Y(n928) );
AND3X1_RVT U996 ( .A1(n930), .A2(n929), .A3(n928), .Y(n932) );
OR2X1_RVT U997 ( .A1(n932), .A2(n931), .Y(n962) );
INVX0_RVT U998 ( .A(n933), .Y(n957) );
OR2X1_RVT U999 ( .A1(n934), .A2(n983), .Y(n936) );
AND2X1_RVT U1000 ( .A1(n936), .A2(n935), .Y(n937) );
OR2X1_RVT U1001 ( .A1(n938), .A2(n937), .Y(n939) );
AND2X1_RVT U1002 ( .A1(n940), .A2(n939), .Y(n941) );
OR2X1_RVT U1003 ( .A1(n942), .A2(n941), .Y(n943) );
AND2X1_RVT U1004 ( .A1(n944), .A2(n943), .Y(n945) );
AND2X1_RVT U1005 ( .A1(n948), .A2(n947), .Y(n949) );
OR2X1_RVT U1006 ( .A1(n950), .A2(n949), .Y(n951) );
AND2X1_RVT U1007 ( .A1(n952), .A2(n951), .Y(n953) );
OR2X1_RVT U1008 ( .A1(n954), .A2(n953), .Y(n955) );
AND3X1_RVT U1009 ( .A1(n957), .A2(n956), .A3(n955), .Y(n958) );
OR3X1_RVT U1010 ( .A1(n960), .A2(n959), .A3(n958), .Y(n961) );
AND4X1_RVT U1011 ( .A1(n964), .A2(n963), .A3(n962), .A4(n961), .Y(n965) );
AND2X1_RVT U1012 ( .A1(n965), .A2(n1022), .Y(n966) );
AND2X1_RVT U1013 ( .A1(opB_reg[0]), .A2(n967), .Y(n968) );
OR2X1_RVT U1014 ( .A1(n969), .A2(n968), .Y(n970) );
OR2X1_RVT U1015 ( .A1(n972), .A2(n971), .Y(n976) );
OR2X1_RVT U1016 ( .A1(n974), .A2(n973), .Y(n975) );
AND3X1_RVT U1017 ( .A1(n977), .A2(n976), .A3(n975), .Y(n980) );
OR2X1_RVT U1018 ( .A1(n382), .A2(n978), .Y(n979) );
AND2X1_RVT U1019 ( .A1(n980), .A2(n979), .Y(n981) );
AND2X1_RVT U1020 ( .A1(opB_reg[0]), .A2(n982), .Y(n989) );
OR2X1_RVT U1021 ( .A1(n984), .A2(n983), .Y(n987) );
```

```
  AND2X1_RVT U1022 ( .A1(n987), .A2(n986), .Y(n988) );
  OR2X1_RVT U1023 ( .A1(n989), .A2(n988), .Y(n990) );
  OR2X1_RVT U1024 ( .A1(n991), .A2(n990), .Y(n998) );
  AND2X1_RVT U1025 ( .A1(n992), .A2(n407), .Y(n994) );
  OR2X1_RVT U1026 ( .A1(n994), .A2(n993), .Y(n995) );
  AND2X1_RVT U1027 ( .A1(n996), .A2(n995), .Y(n997) );
  OR2X1_RVT U1028 ( .A1(n998), .A2(n997), .Y(n1000) );
  AND2X1_RVT U1029 ( .A1(n1000), .A2(n999), .Y(n1002) );
  OR2X1_RVT U1030 ( .A1(n1002), .A2(n1001), .Y(n309) );
  NBUFFX2_RVT U1031 ( .A(n1003), .Y(n1029) );
  NBUFFX2_RVT U1032 ( .A(n1003), .Y(n1030) );
  NBUFFX2_RVT U1033 ( .A(n1003), .Y(n1031) );
endmodule
```